

LAPORAN MILESTONE 2 - SYNTAX ANALYSIS
TUGAS BESAR ARION COMPILER
IF2224 TEORI BAHASA FORMAL DAN AUTOMATA



Disusun oleh:

K1 - Kelompok 13: ToBeFairO

13524003 - Faiq Azzam Nafidz
13524013 - Anindya Naufal Pinasthika
13524083 - Mahmudia Kimdaro Amin
13524095 - Jingglang Galih Rinenggan

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

1. Teori Singkat.....	2
1.1. Definisi dan Peran Parser.....	2
1.2. Keterkaitan dengan Lexer.....	2
1.3. Algoritma Recursive Descent.....	2
1.4. Parse Tree.....	3
2. Perancangan & Implementasi.....	4
2.1. Implementasi Kelas Node.....	9
2.2. Implementasi Kelas Parser.....	13
3. Pengujian.....	19
3.1. Test Case 1.....	19
3.2. Test Case 2.....	22
3.3. Test Case 3.....	25
3.4. Test Case 4.....	29
3.5. Test Case 5.....	35
3.6. Test Case 6.....	46
3.7. Test Case 7.....	51
3.8. Test Case 8.....	55
3.9. Test Case 9.....	57
4. Kesimpulan dan Saran.....	60
4.1. Kesimpulan.....	60
4.2. Saran.....	60
5. Lampiran.....	61
5.1. Github.....	61
5.2. Tabel Pembagian Tugas.....	61
6. Referensi.....	62

1. Teori Singkat

1.1. Definisi dan Peran Parser

Parser atau *syntax analysis* adalah salah satu tahap proses dari banyaknya rangkaian proses kompilasi sebuah kode. Parser akan membangun struktur sintaks seperti *parse tree* atau *AST (Abstract Syntax Tree)* yang merepresentasikan struktur hierarkis token-token masukan yang merepresentasikan program sumber berdasarkan grammar yang telah didefinisikan.

Parser bertugas untuk memeriksa urutan token yang dihasilkan oleh *lexical analyzer* agar sesuai dengan aturan tata bahasa atau *grammar* dari bahasa pemrograman yang dikompilasi dan meneruskan representasi program ke tahap proses kompilasi berikutnya.

1.2. Keterkaitan dengan Lexer

Parser merupakan tahap kompilasi berikutnya ketika sebuah kode program telah di-*tokenize* oleh lexical analysis (Lexer). Jika lexical analysis menjawab pertanyaan "apa token-token dari kode ini?", maka selanjutnya setelah didefinisikan token-token oleh Lexer, syntax analysis menjawab pertanyaan "apakah susunan token ini valid?"

Di tahap inilah suatu *compiler/interpreter* mengetahui kesalahan *syntax* suatu kode program. Suatu parser dapat menentukan apakah susunan token yang dihasilkan oleh *lexical analyzer* adalah valid atau tidak. Jika ditentukan bahwa susunan tersebut valid, suatu parser akan menghasilkan struktur hierarkis program berupa *parse tree*.

1.3. Algoritma Recursive Descent

Recursive Descent adalah teknik parsing top-down yang menggunakan sekumpulan prosedur/fungsi rekursif untuk memproses input berupa daftar token yang berurutan. Setiap non-terminal dalam *grammar* diimplementasikan sebagai satu fungsi.

Cara kerja recursive descent adalah algoritma akan memanggil fungsi produksi awal yang didefinisikan pada *grammar* yang berlaku. Fungsi tersebut akan memanggil fungsi produksi lainnya atau mengecek apakah token yang sedang dibaca saat itu sesuai dengan ekspektasi program, tergantung pada *grammar*. Algoritma akan terus

berjalan sampai produksi pertama, yaitu produksi simpul Program, selesai diproduksi, baik berhasil diproduksi atau gagal diproduksi. Simpul program yang berhasil diproduksi memiliki arti bahwa susunan token yang dibaca valid, sedangkan simpul program yang gagal diproduksi berarti susunan token tidak valid.

1.4. Parse Tree

Parse tree, atau *Concrete Syntax Tree* adalah representasi lengkap dari derivation yang menunjukkan setiap aturan produksi atau *grammar* yang digunakan. Akar dari sebuah *parse tree* merepresentasikan variabel awal pada produksi sesuai *grammar*. Parser dapat menggunakan algoritma *recursive descent* untuk memproduksi setiap simpul yang membentuk *parse tree*.

Setiap produksi yang berhasil dilakukan oleh suatu parser akan masuk ke dalam kumpulan simpul pada *parse tree* sebagai anak dari simpul sebelumnya (kecuali simpul akar yang tidak memiliki *parent*). Dengan begitu, suatu parser dapat membentuk *parse tree* dengan mencoba untuk membentuk simpul akar saja, karena simpul akar akan mencoba membentuk simpul anak-anaknya, dan simpul anak-anaknya akan mencoba membentuk simpul selanjutnya sampai *parse tree* terbentuk penuh atau *parse tree* gagal dibentuk.

2. Perancangan & Implementasi

Program dibuat dengan mendefinisikan terlebih dahulu struktur data simpul dari *parse tree* yang akan dibuat program. Struktur data simpul diimplementasikan dalam kelas Node yang berisi atribut type bertipe NodeType, token bertipe Token, dan children bertipe vector<Node*>. Variabel type mendefinisikan tipe simpul yang bisa bertipe sesuai dengan nama *production rule* yang berlaku pada *grammar* terkait atau simpul bertipe terminal (merepresentasikan token). Variabel Token bisa berisi token kosong untuk simpul bukan terminal atau berisi token yang sesuai dengan *production rule* pada simpul terminal, dengan tipe Token menggunakan definisi yang ada pada *milestone* 1. Variabel Children menyimpan daftar *pointer* yang menunjuk ke anak-anak dari simpul tersebut yang merupakan hasil dari berhasilnya fungsi produksi memproduksi anak-anak simpul sesuai dengan *production rule*. Terdapat juga fungsi-fungsi tambahan untuk mengonstruksi objek, menghancurkan objek, membuat *string* berdasarkan objek, dan mengonstruksi *string* yang merepresentasikan *parse tree* yang terbuat.

Inti program berada pada fungsi parser yang diimplementasikan dalam kelas Parser. Kelas ini memiliki atribut tokens bertipe vector<Token> (daftar token yang akan dibaca), expected bertipe set<string> (daftar nama token yang diharapkan program di suatu waktu), pos bertipe int (posisi token pada tokens yang sedang dibaca), highestPos bertipe int (posisi token tertinggi yang sudah pernah terbaca), root bertipe Node* (menunjuk akar dari *parse tree* yang terbuat), dan curr bertipe Node* (menunjuk simpul yang sedang dilakukan proses produksi). Atribut-atribut ini digunakan untuk membantu pemrosesan *parsing* yang dilakukan di dalam kelas.

Beberapa *method-method* juga perlu didefinisikan pada kelas Parser, seperti peek(), pop(), success(), fails(), restore(), insert(), dan match().

- Fungsi peek() mengembalikan token yang sedang dibaca tanpa menambah nilai pos.
- Fungsi pop() sama seperti fungsi peek() namun menambah nilai pos sebanyak satu.
- Fungsi success() dipanggil ketika suatu produksi berhasil, fungsi menambahkan simpul terbuat ke dalam atribut children dari *parent* simpul.

- Fungsi `fails()` dipanggil ketika suatu produksi gagal, fungsi hanya mengembalikan nilai `curr` menjadi *pointer* yang menunjuk *parent*.
- Fungsi `restore()` dipanggil saat ingin melakukan suatu produksi sehingga nilai `pos` kembali ke nilai yang dimasukkan dalam fungsi tersebut. Dengan ini, jika salah satu aturan produksi gagal di tengah (setelah mengonsumsi beberapa token), suatu fungsi produksi bisa mengembalikan posisi token yang ingin dibaca. Fungsi juga memperbarui nilai `highestPos` dengan nilai `pos` jika `pos` saat itu lebih besar dari nilai `highestPos`.
- Fungsi `insert()` dipanggil di awal produksi untuk membuat simpul baru sesuai dengan tipe yang dimasukkan dalam argumen, mengganti nilai `curr` untuk menunjuk simpul baru tersebut, dan mengembalikan nilai `curr` sebelumnya.
- Fungsi `match()` digunakan untuk mengecek token yang sedang dibaca sama dengan nama token yang dimasukkan ke argumen. Fungsi ini akan menambah nilai `pos` dan mengembalikan nilai *true* jika token benar dan mengembalikan nilai *false* saja jika salah. Saat token salah, fungsi ini juga menambahkan nama token yang ada di argumen ke dalam `expected` jika nilai `pos` saat itu sama dengan nilai `highestPos` atau meng-*overwrite* `expected` dengan satu nama token tersebut jika nilai `pos` lebih besar dari `highestPos`. Hal ini dilakukan agar kelas dapat menyimpan nama-nama token yang diharapkan berdasarkan aturan produksi yang ada pada token yang dibaca di paling akhir.

Production rule yang berlaku diimplementasi oleh *method-method* yang merepresentasikan produksi setiap simpul. Fungsi produksi tersebut bekerja dengan memanggil produksi simpul lain atau memanggil method `match()` untuk mengecek token sesuai dengan aturan produksi yang berlaku. Umumnya, fungsi produksi berbentuk seperti berikut.

```
bool Parser::<nodeType>Prod()
{
    Node *parent = insert(<nodeType>);
    int save = pos;

    if (<production rule 1>)
        return success(parent);
    restore(save);
}
```

```

    if (<production rule 2>)
        return success(parent);
    restore(save);
    if (<production rule 3 part 1>)
    {
        int save = pos;
        if (<production rule 3 part 2>)
            return success(parent);
        restore(save);
    }
    restore(save);
    ...

    return fails(parent);
}

```

Sebagai contoh, terdapat aturan produksi $\langle \text{array-type} \rangle \rightarrow \text{arraysy} + \text{lbrack} + (\text{range} \mid \text{ident}) + \text{rbrack} + \text{ofsy} + \text{type}$, sehingga fungsi produksinya seperti berikut.

```

bool Parser::arrayTypeProd()
{
    Node *parent = insert(array_type);
    int save = pos;

    if (match("arraysy") && match("lbrack") && (match("ident") ||
rangeProd()) && match("rbrack") && match("ofsy") && typeProd())
        return success(parent);

    restore(save);
    return fails(parent);
}

```

Aturan produksi untuk setiap tipe simpul mengikuti aturan *grammar* pada tabel berikut.

Variabel	Aturan Produksi
$\langle \text{program} \rangle$	$\langle \text{program-header} \rangle \langle \text{declaration-part} \rangle$ $\langle \text{compound-statement} \rangle \text{ period}$
$\langle \text{program-header} \rangle$	$\text{programsy ident semicolon}$
$\langle \text{declaration-part} \rangle$	$\langle \text{const-declaration} \rangle \langle \text{declaration-part} \rangle \mid$

	<type-declaration> <declaration-part> <var-declaration> <declaration-part> <subprogram-declaration> <declaration-part> epsilon
<const-declaration>	constsy ident eql <constant> semicolon (const-list)
(more-const-declaration)	ident eql <constant> semicolon (const-list) epsilon
<constant>	charcon string (signed-const)
(signed-const)	(sign) (unsigned-const)
(sign)	plus minus epsilon
(unsigned-const)	ident intcon realcon
<type-declaration>	typesy (type-list)
(type-list)	ident eql <type> semicolon (type-list-tail)
(type-list-tail)	ident eql <type> semicolon (type-list-tail) epsilon
<var-declaration>	varsy (var-list)
(var-list)	<identifier-list> colon <type> semicolon (var-list-tail)
(var-list-tail)	<identifier-list> colon <type> semicolon (var-list-tail) epsilon
<identifier-list>	ident (ident-list-tail)
(ident-list-tail)	comma ident (ident-list-tail) epsilon
<type>	ident <array-type> <range> <enumerated> <record-type>
<array-type>	arraysy lbrack (array-index) rbrack ofsy <type>
(array-index)	<range> ident
<range>	<constant> period period <constant>
<enumerated>	lparent ident (ident-list-tail) rparent
<record-type>	recordsy <field-list> endsy
<field-list>	<field-part> (field-list-tail)
(field-list-tail)	semicolon <field-part> (field-list-tail) epsilon
<field-part>	<identifier-list> colon <type>
<subprogram-declaration>	<procedure-declaration> <function-declaration>
<procedure-declaration>	proceduresy ident (formal-param-opt) semicolon <block> semicolon
(formal-param-opt)	<formal-parameter-list> epsilon
<function-declaration>	functionsy ident (formal-param-opt) colon ident semicolon <block> semicolon

<block>	<declaration-part> <compound-statement>
<formal-parameter-list>	lparent <parameter-group> (param-group-tail) rparent
(param-group-tail)	semicolon <parameter-group> (param-group-tail) epsilon
<parameter-group>	<identifier-list> colon (param-type)
(param-type)	ident <array-type>
<compound-statement>	beginsy <statement-list> endsy
<statement-list>	<statement> (stmt-list-tail)
(stmt-list-tail)	semicolon <statement> (stmt-list-tail) epsilon
<statement>	<assignment-statement> <if-statement> <case-statement> <while-statement> <repeat-statement> <for-statement> <procedure/function-call> epsilon
<variable>	ident (comp-var-list)
(comp-var-list)	<component-variable> (comp-var-list) epsilon
<component-variable>	lbrack <index-list> rbrack period ident
<index-list>	(index-elem) (index-list-tail)
(index-elem)	intcon charcon ident
(index-list-tail)	comma (index-elem) (index-list-tail) epsilon
<assignment-statement>	<variable> becomes <expression>
<if-statement>	ifsy <expression> thensy <statement> (else-opt)
(else-opt)	elsy <statement> epsilon
<case-statement>	casesy <expression> ofsy <case-block> endsy
<case-block>	<case-labels> colon <statement> (case-block-tail)
<case-labels>	<constant> (case-label-tail)
(case-label-tail)	comma <constant> (case-label-tail) epsilon
(case-block-tail)	semicolon (case-block-opt) (case-block-tail) epsilon
(case-block-opt)	<case-block> epsilon
<while-statement>	whilesy <expression> dosy <compound-statement>
<repeat-statement>	repeatsy <statement-list> untilsy <expression>
<for-statement>	forsy ident becomes <expression> (for-dir) <expression> dosy <compound-statement>

(for-dir)	tosy downtosy
<procedure/function-call>	ident (actual-param-opt)
(actual-param-opt)	lparent (param-list-opt) rparent epsilon
(param-list-opt)	<parameter-list> epsilon
<parameter-list>	<expression> (expr-list-tail)
(expr-list-tail)	comma <expression> (expr-list-tail) epsilon
<expression>	<simple-expression> (rel-op-expr-opt)
(rel-op-expr-opt)	<relational-operator> <simple-expression> epsilon
<simple-expression>	(sign-option) <term> (add-op-term-list)
(sign-option)	plus minus epsilon
(add-op-term-list)	<additive-operator> <term> (add-op-term-list) epsilon
<term>	<factor> (mul-op-factor-list)
(mul-op-factor-list)	<multiplicative-operator> <factor> (mul-op-factor-list) epsilon
<factor>	ident intcon realcon charcon string lparent <expression> rparent notsy <factor> <procedure/function-call> <variable>
<relational-operator>	eql neq gtr geq lss leq
<additive-operator>	plus minus orsy
<multiplicative-operator>	times rdiv idiv imod andsy

Catatan:

- Variabel yang diapit dengan “<” merupakan tipe simpul yang akan muncul pada *parse tree*.
- Variabel yang diapit dengan “()” adalah variabel tambahan sebagai pembantu pembuatan aturan produksi tipe simpul yang ada.
- Kata yang tidak diapit baik dengan “<” maupun “()” adalah terminal.

Berikut adalah implementasi dari kelas Node dan kelas Parser.

2.1. Implementasi Kelas Node

Node.hpp

```
#pragma once
#include "../Lexer/Lexer.hpp"
#include <iostream>
```

```
enum NodeType {
    program,
    program_header,
    declaration_part,
    const_declaration,
    constant,
    type_declaration,
    var_declaration,
    identifier_list,
    type,
    array_type,
    range,
    enumerated,
    record_type,
    field_list,
    field_part,
    subprogram_declaration,
    procedure_declaration,
    function_declaration,
    block,
    formal_parameter_list,
    parameter_group,
    compound_statement,
    statement_list,
    statement,
    variable,
    component_variable,
    index_list,
    assignment_statement,
    if_statement,
    case_statement,
    case_block,
    while_statement,
    repeat_statement,
    for_statement,
    procedure_function_call,
    parameter_list,
    expression,
    simple_expression,
    term,
    factor,
```

```

    relational_operator,
    additive_operator,
    multiplicative_operator,
    terminal
};

const std::string nodeTypeStr[] = {
    "<program>",
    "<program-header>",
    "<declaration-part>",
    "<const-declaration>",
    "<constant>",
    "<type-declaration>",
    "<var-declaration>",
    "<identifier-list>",
    "<type>",
    "<array-type>",
    "<range>",
    "<enumerated>",
    "<record-type>",
    "<field-list>",
    "<field-part>",
    "<subprogram-declaration>",
    "<procedure-declaration>",
    "<function-declaration>",
    "block",
    "<formal-parameter-list>",
    "<parameter-group>",
    "<compound-statement>",
    "<statement-list>",
    "<statement>",
    "<variable>",
    "<component_variable>",
    "<index_list>",
    "<assignment-statement>",
    "<if-statement>",
    "<case-statement>",
    "<case-block>",
    "<while-statement>",
    "<repeat-statement>",
    "<for-statement>",

```

```

    "<procedure/function-call>",
    "<parameter-list>",
    "<expression>",
    "<simple-expression>",
    "<term>",
    "<factor>",
    "<relational-operator>",
    "<additive-operator>",
    "<multiplicative-operator>",
    "<terminal>"
};

class Node {
public:
    NodeType type;
    Token token;
    vector<Node*> children;

    Node(NodeType type): type(type), token(Token("", "")) {}
    Node(Token token): type(terminal), token(token) {}
    virtual ~Node() {
        for(Node* child : children) {
            delete child;
        }
    }
    string toString() {
        if(type == terminal) return token.toString();
        else return nodeTypeStr[type];
    }

    void printTreeToFile(std::string fileName);
    void printTree(std::ostream *stream, int depth, bool isLast,
vector<bool> isAboveLeaf);
};

```

Node.cpp

```

#include "Node.hpp"

void Node::printTreeToFile(std::string fileName) {

```

```

        ofstream file("test/Parser/" + fileName);

        vector<bool> isAboveLeaf;
        printTree(&file, 0, true, isAboveLeaf);

        file.close();
    }

    void Node::printTree(std::ostream* stream, int depth, bool
isLast, vector<bool> isAboveLeaf) {
        while (isAboveLeaf.size() < (size_t) depth)
isAboveLeaf.push_back(isLast);
        for(int i = 0; i < depth; i++) {
            if(i+1 == depth) {
                if(!isLast) *stream << "├─ ";
                else *stream << "└─ ";
            }
            else {
                if(!isAboveLeaf.at(i)) *stream << "| ";
                else *stream << " ";
            }
        }
        *stream << toString() << "\n";
        // cout << toString();
        for(size_t i = 0; i < children.size(); i++) {
            if(i == children.size() - 1) isLast = true;
            else isLast = false;
            children.at(i)->printTree(stream, depth+1, isLast,
isAboveLeaf);
        }
    }
}

```

2.2. Implementasi Kelas Parser

Parser.hpp

```

#pragma once
#include "Node.hpp"
#include <set>

class Parser
{

```

```

private:
    vector<Token> tokens;
    set<string> expected;
    int pos;
    int highestPos;
    Node *root;
    Node *curr;

    // Main function

    // Return token at pos value
    Token peek();
    // "Consume" token by incrementing pos and return token
    before incrementing
    Token pop();
    // Called when a production succeeded, appending the curr
    node to the parent node and replace curr with parent, always
    returns true
    bool success(Node *parent);
    // Called when a production fails, replaces curr with
    parent, always return false
    bool fails(Node *parent);
    // Called when part of a production fails, replaces the pos
    value to save
    void restore(int save);
    // Called at the beginning of a production, creating a node
    with type specified in the argument, replaces curr with node
    created, and returns the previous curr
    Node *insert(NodeType type);
    // To check whether the token being read (at pos) have the
    same name as t
    bool match(string t);

    // Production function

    // program-header + declaration-part + compound-statement +
    period
    bool programProd();
    // programsy + ident + semicolon
    bool programHeaderProd();
    // (const-declaration)* + (type-declaration)* +

```

```

(var-declaration)* + (subprogram-declaration)*
    bool declarationPartProd();
    // constsy + (ident + eql + constant + semicolon)+
...
    // plus | minus | orsy
    bool additiveOperatorProd();
    // times | rdiv | idiv | imod | andsy
    bool multiplicativeOperatorProd();

public:
    Parser(vector<Token> tokens) : tokens(tokens), pos(0),
highestPos(0) {}

    bool parse();

    Node *getRoot() const { return root; }
    vector<Token> getTokens() const { return tokens; }
    set<string> getExpected() const { return expected; }
    int getPos() const { return pos; }
    int getHighestPos() const { return highestPos; }
};

```

Parser.cpp

```

#include "Parser.hpp"

Token Parser::peek()
{
    return tokens[pos];
}

Token Parser::pop()
{
    return tokens[pos++];
}

bool Parser::success(Node *parent)
{
    parent->children.push_back(curr);
    curr = parent;
    return true;
}

```



```

}

bool Parser::fails(Node *parent)
{
    delete curr;
    curr = parent;
    return false;
}

void Parser::backTrack(int save)
{
    if (highestPos < pos)
        highestPos = pos;
    pos = save;
}

Node *Parser::insert(NodeType type)
{
    Node *parent = curr;
    Node *node = new Node(type);
    curr = node;
    return parent;
}

bool Parser::match(string t)
{
    while (peek().getType() == "comment")
        pop();
    if (peek().getType() == t)
    {
        Node *node = new Node(tokens[pos]);
        pop();
        curr->children.push_back(node);
        return true;
    }

    if (pos > highestPos)
    {
        expected.clear();
        expected.insert(t);
    }
}

```

```

        else if (pos == highestPos)
        {
            expected.insert(t);
        }

        return false;
    }

    bool Parser::parse()
    {
        root = nullptr;
        bool result = programProd();
        root = curr;
        return result;
    }

    bool Parser::programProd()
    {
        curr = new Node(program);
        int save = pos;

        if (programHeaderProd() && declarationPartProd() &&
            compoundStatementProd() && match("period"))
            return true;

        backTrack(save);
        delete curr;
        curr = nullptr;
        return false;
    }

    bool Parser::programHeaderProd()
    {
        Node *parent = insert(program_header);
        int save = pos;

        if (match("programsy") && match("ident") &&
            match("semicolon"))
            return success(parent);

        backTrack(save);
    }

```

```

        return fails(parent);
    }

bool Parser::declarationPartProd()
{
    Node *parent = insert(declaration_part);
    int save = pos;

    while (constDeclarationProd())
        save = pos;
    backtrack(save);
    while (typeDeclarationProd())
        save = pos;
    backtrack(save);
    while (varDeclarationProd())
        save = pos;
    backtrack(save);
    while (subprogramDeclarationProd())
        save = pos;
    backtrack(save);

    return success(parent);
}

...
bool Parser::additiveOperatorProd()
{
    Node *parent = insert(additive_operator);
    int save = pos;

    if (match("plus"))
        return success(parent);
    backtrack(save);
    if (match("minus"))
        return success(parent);
    backtrack(save);
    if (match("orsy"))
        return success(parent);
    backtrack(save);
    return fails(parent);
}

```

```

bool Parser::multiplicativeOperatorProd()
{
    Node *parent = insert(multiplicative_operator);
    int save = pos;

    if (match("times"))
        return success(parent);
    backTrack(save);
    if (match("rdiv"))
        return success(parent);
    backTrack(save);
    if (match("idiv"))
        return success(parent);
    backTrack(save);
    if (match("imod"))
        return success(parent);
    backTrack(save);
    if (match("andsy"))
        return success(parent);
    backTrack(save);
    return fails(parent);
}

```

3. Pengujian

3.1. Test Case 1

```

program Hello;

var
    a, b: integer;

begin
    a := 5;
    b := a + 10;
    writeln('Result = ', b);
end.

```

```

programsy
ident (Hello)
semicolon
varsy

```

```

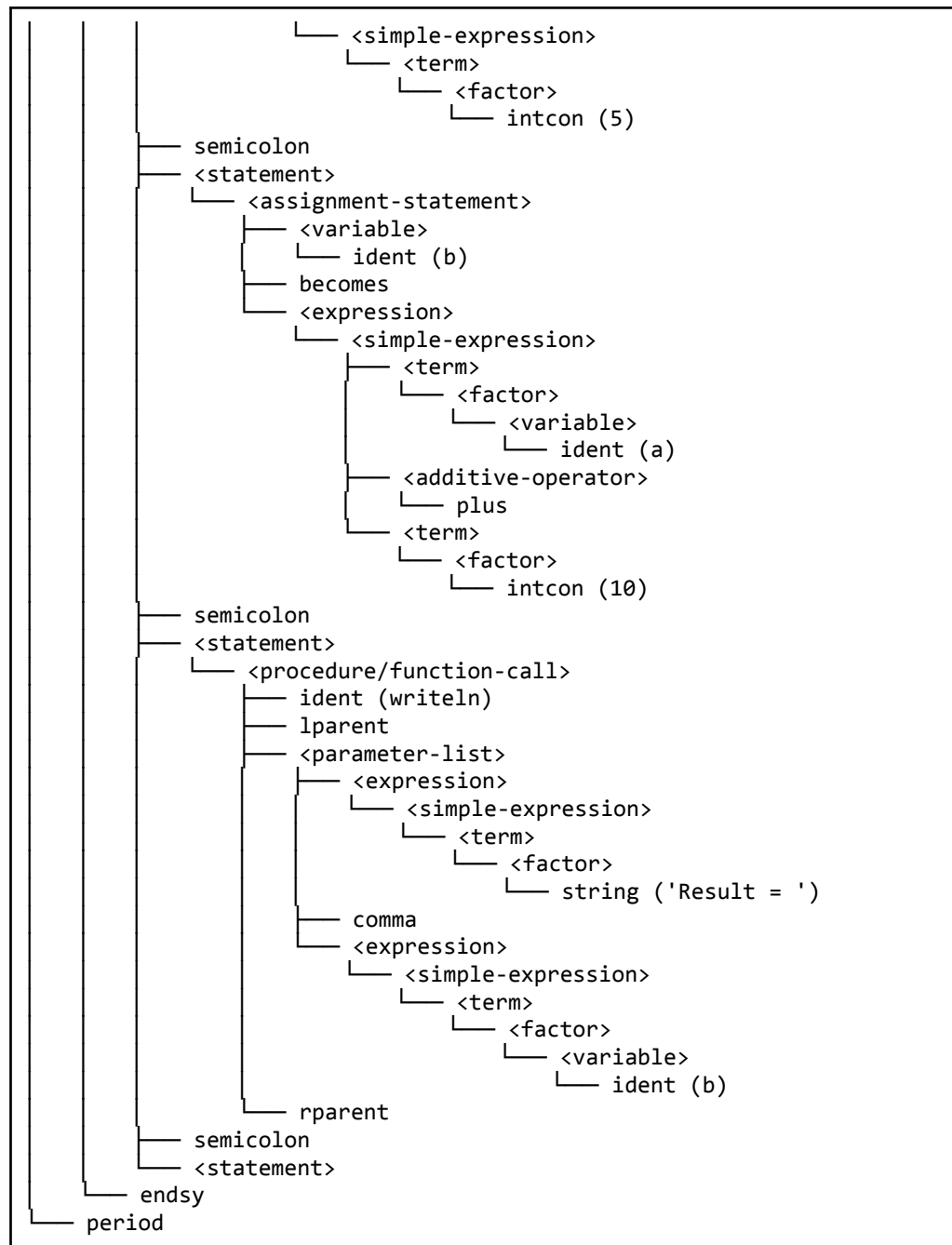
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
ident (writeln)
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon
endsy
period

```

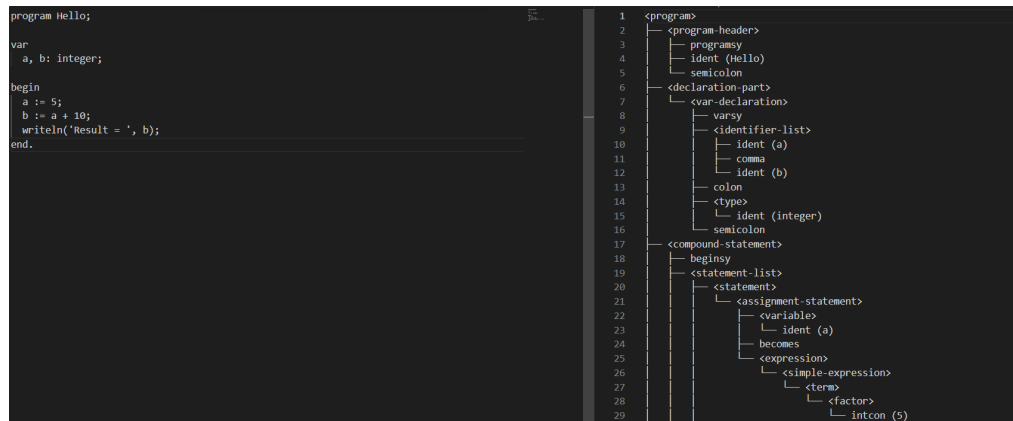
```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Hello)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   └── ident (b)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
└── <compound-statement>
    ├── beginsy
    ├── <statement-list>
    │   └── <statement>
    │       └── <assignment-statement>
    │           ├── <variable>
    │           │   └── ident (a)
    │           ├── becomes
    │           └── <expression>

```



Bukti *screenshot*:



Gambar 1. Input dan Output Test Case 1

3.2. Test Case 2

```

program ControlFlow;
var
  i: integer;
begin
  i := 10;
  while i > 0 do
  begin
    if i mod 5 == 0 then
      writeln(i);
    i := i - 1;
  end
end.

```

```

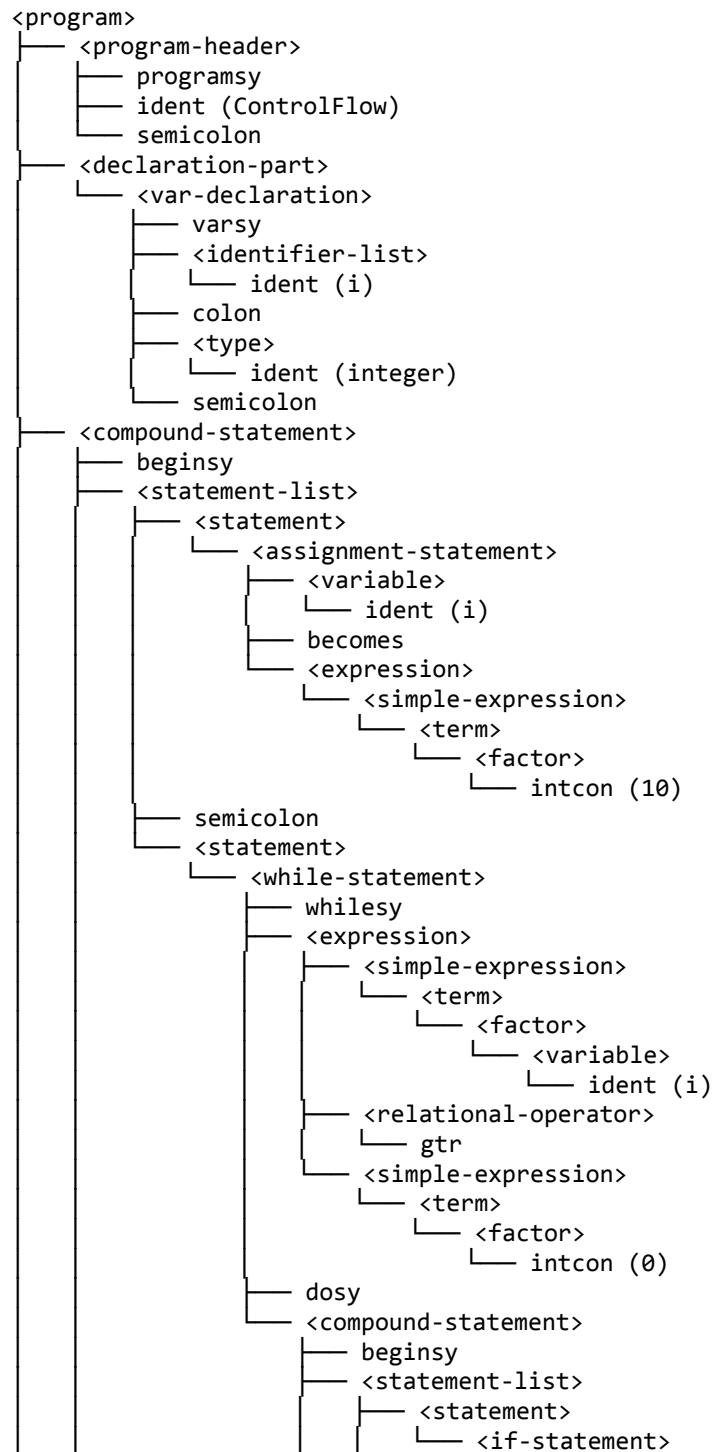
programsy
ident (ControlFlow)
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
beginsy
ident (i)
becomes
intcon (10)
semicolon
whilesy
ident (i)
gtr
intcon (0)
dosy
beginsy
ifsy
ident (i)
imod
intcon (5)
eq
intcon (0)
thensy
ident (writeln)
lparent
ident (i)

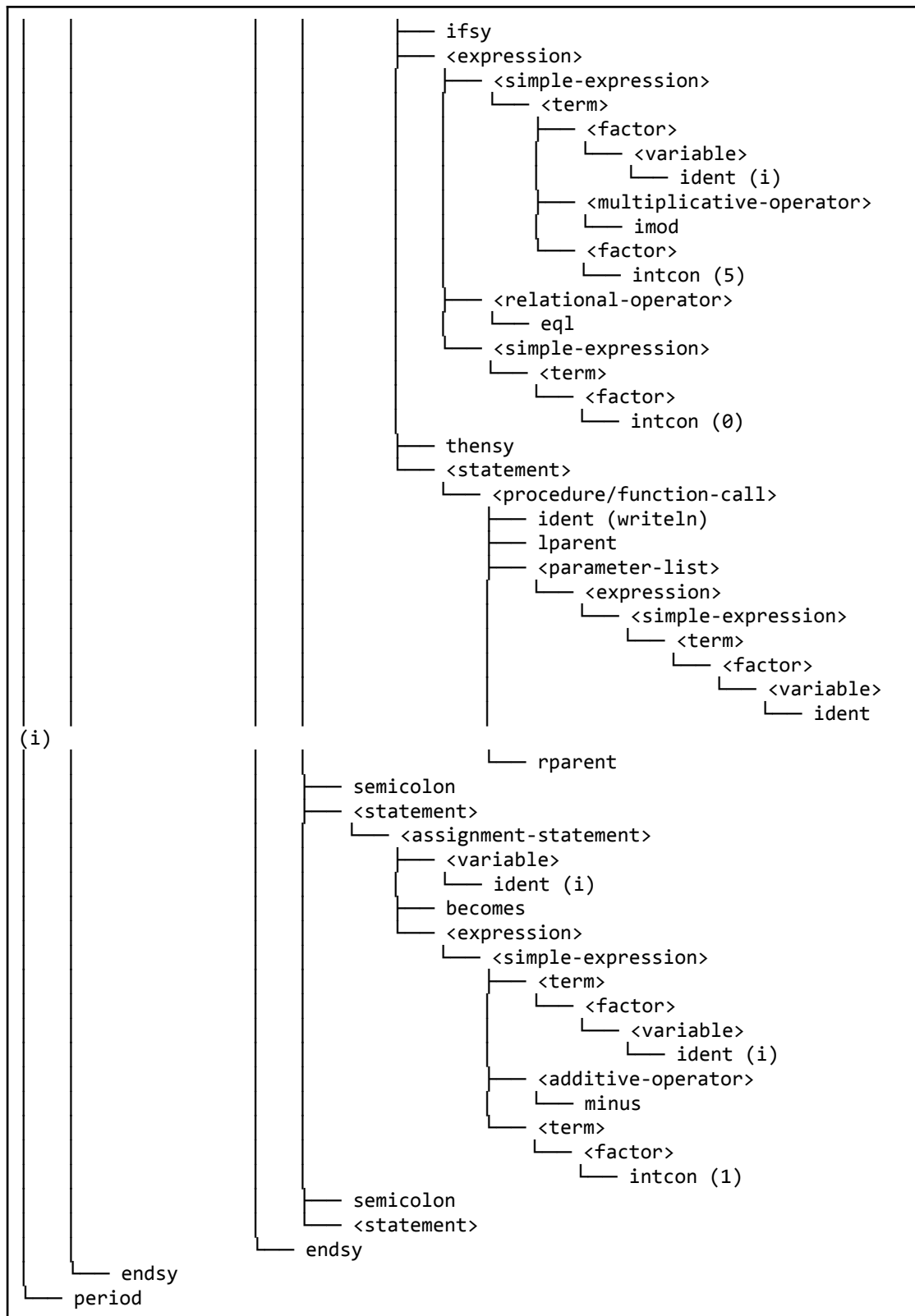
```

```

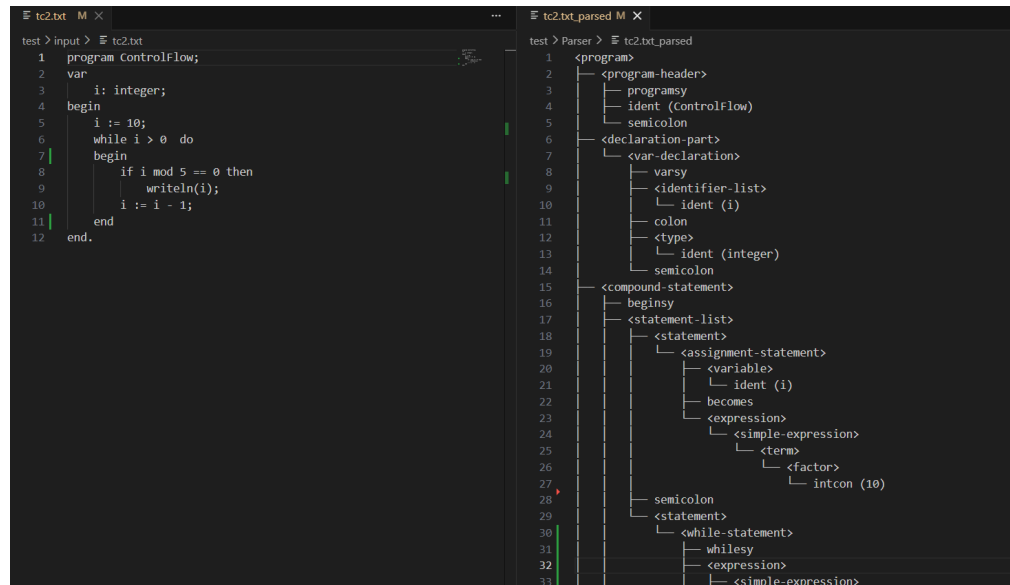
rparent
semicolon
ident (i)
becomes
ident (i)
minus
intcon (1)
semicolon
endsy
endsy
period

```





Bukti screenshot:



Gambar 2. Input dan Output Test Case 2

3.3. Test Case 3

```

program testingagain;

const
  PI == 3.14;
  NumOfiteratior == 100;
  Pesan == 'Selamat Datang';
  BeeMovieScript == 'According to all all law';

var
  diameter, r: Real;
  hate: Boolean;

begin
  r := 12.7;
  area := PI * (r*r);
  hate := False;
  writeln(BeeMovieScript);
  writeln('Area of circle: ');
  writeln(area)
end.

```

```

programsy
ident (testingagain)
semicolon
constsy
ident (PI)
eq1
realcon (3.14)
semicolon
ident (NumOfiteratior)
eq1
intcon (100)
semicolon
ident (Pesan)
eq1
string ('Selamat Datang')
semicolon

```

```

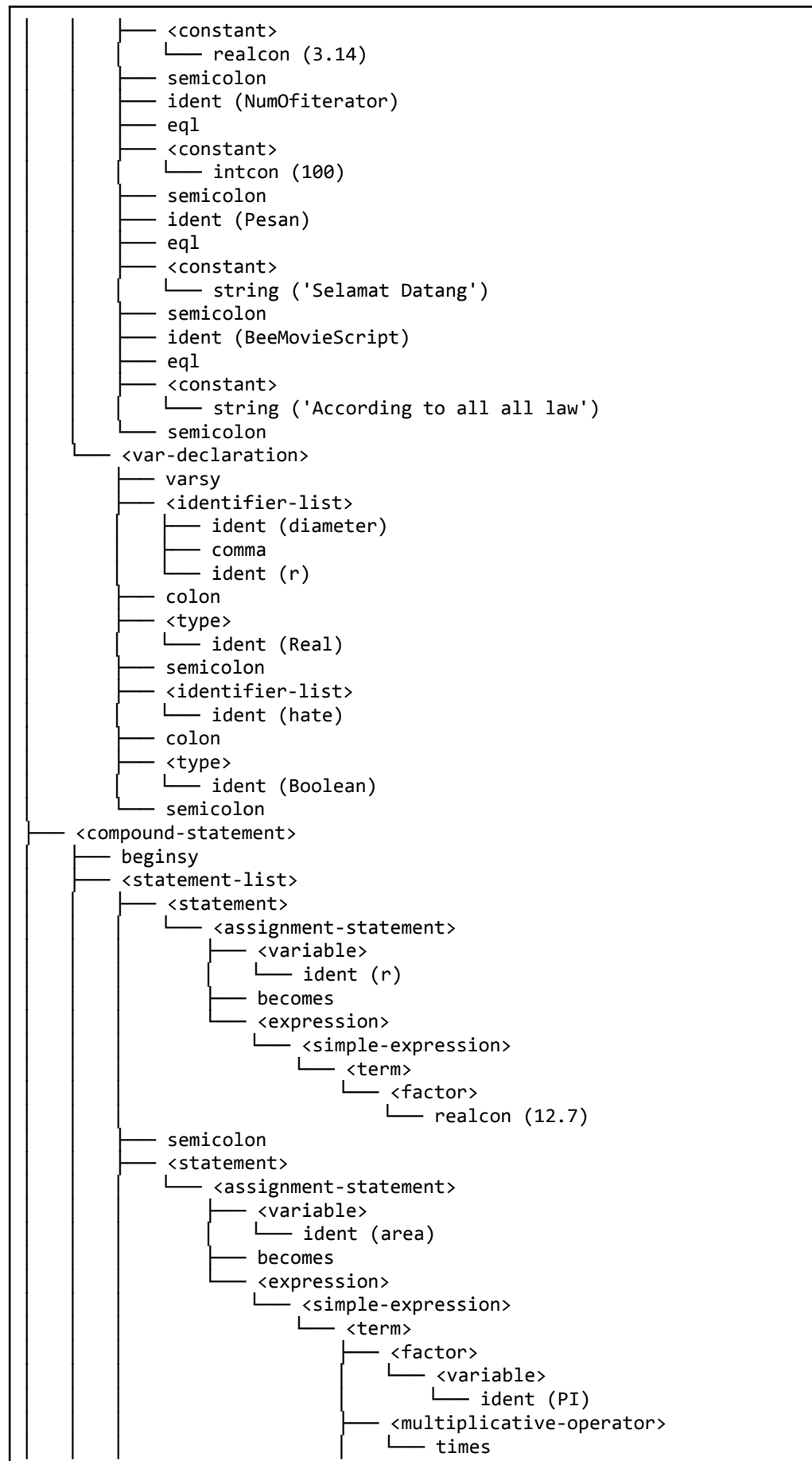
ident (BeeMovieScript)
eq1
string ('According to all all law')
semicolon
varsy
ident (diameter)
comma
ident (r)
colon
ident (Real)
semicolon
ident (hate)
colon
ident (Boolean)
semicolon
beginsy
ident (r)
becomes
realcon (12.7)
semicolon
ident (area)
becomes
ident (PI)
times
lparent
ident (r)
times
ident (r)
rparent
semicolon
ident (hate)
becomes
ident (False)
semicolon
ident (writeln)
lparent
ident (BeeMovieScript)
rparent
semicolon
ident (writeln)
lparent
string ('Area of circle: ')
rparent
semicolon
ident (writeln)
lparent
ident (area)
rparent
endsy
period

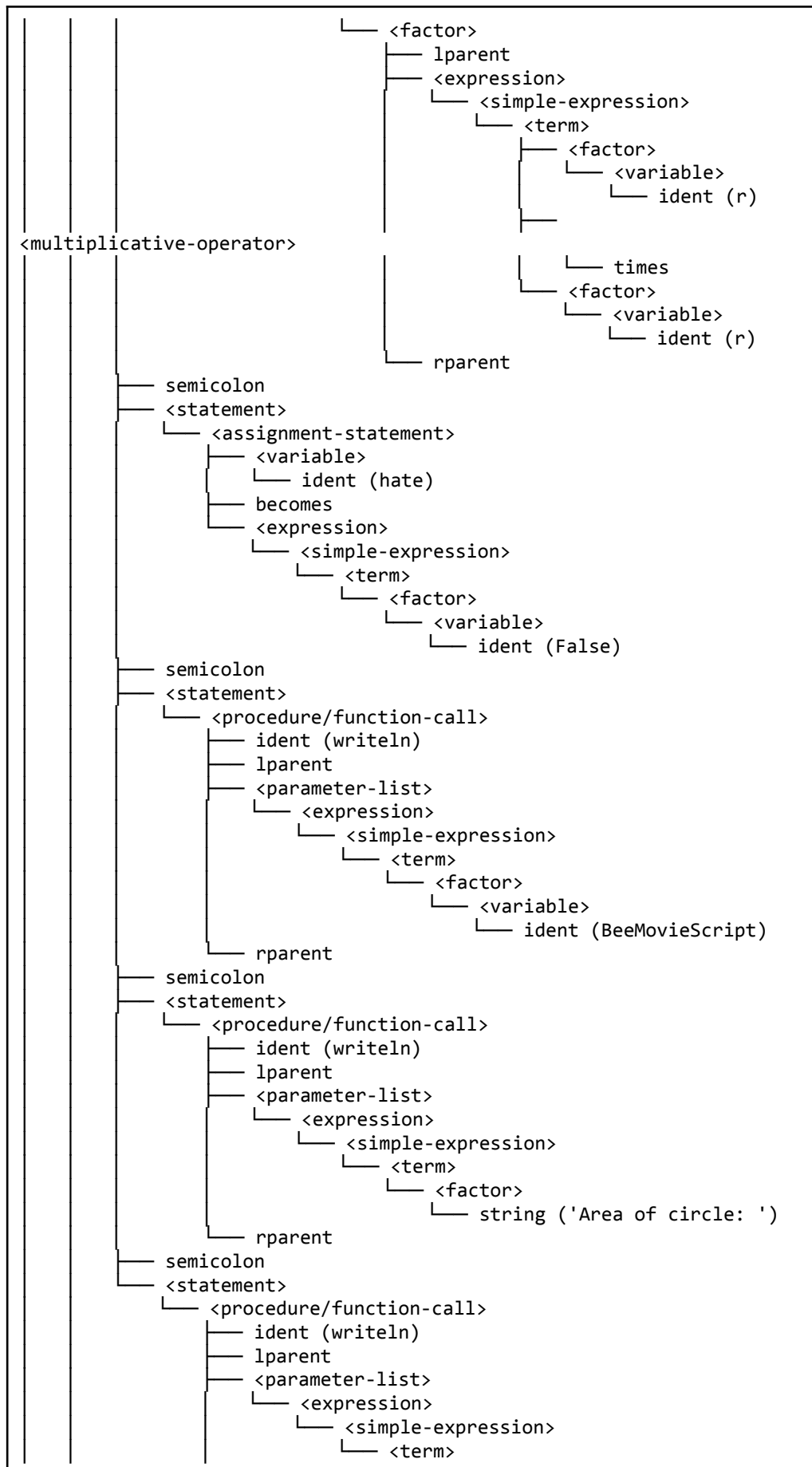
```

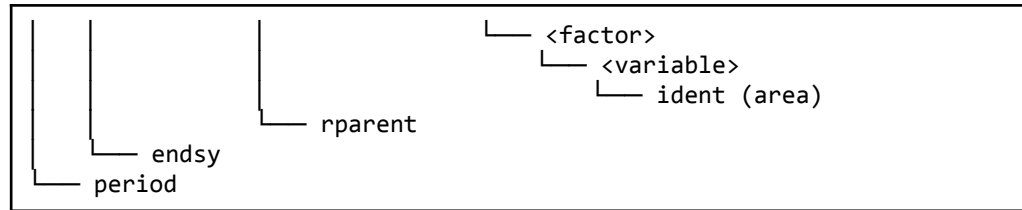
```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (testingagain)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── ident (PI)
│   │   └── eq1

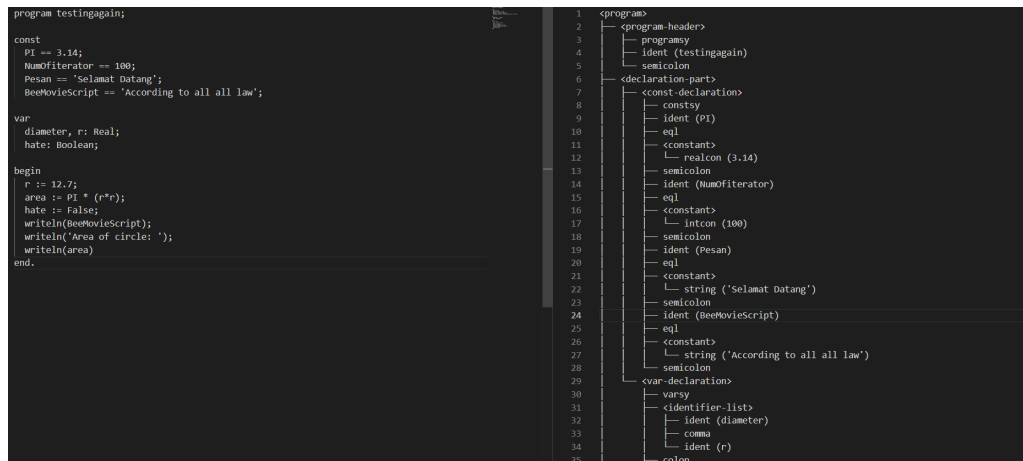
```







Bukti screenshot:



Gambar 3. Input dan Output Test Case 3

3.4. Test Case 4

```

program OddvsEven;

var
  i, limit, level: Integer;
  choices: Char;

begin
  limit := 5;

  for i := 1 to limit do
    if (i mod 2 == 0) then
      writeln('Genap')
    else
      writeln('Ganjil');

  while i < 10 do
    i := i + 1;

  repeat
    limit := limit - 1;
  until limit == 0;

  case choices of
    'A': level := 100;
    'B': level := 50;
    'C': level := 10
  end;

;
end.

```

```
programsy
ident (OddvsEven)
semicolon
varsy
ident (i)
comma
ident (limit)
comma
ident (level)
colon
ident (Integer)
semicolon
ident (choices)
colon
ident (Char)
semicolon
beginsy
ident (limit)
becomes
intcon (5)
semicolon
forsy
ident (i)
becomes
intcon (1)
tosy
ident (limit)
dosy
ifsy
lparent
ident (i)
imod
intcon (2)
eq1
intcon (0)
rparent
thensy
ident (writeln)
lparent
string ('Genap')
rparent
elsesy
ident (writeln)
lparent
string ('Ganjil')
rparent
semicolon
whilesy
ident (i)
lss
intcon (10)
dosy
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon
repeatsy
ident (limit)
becomes
```

```

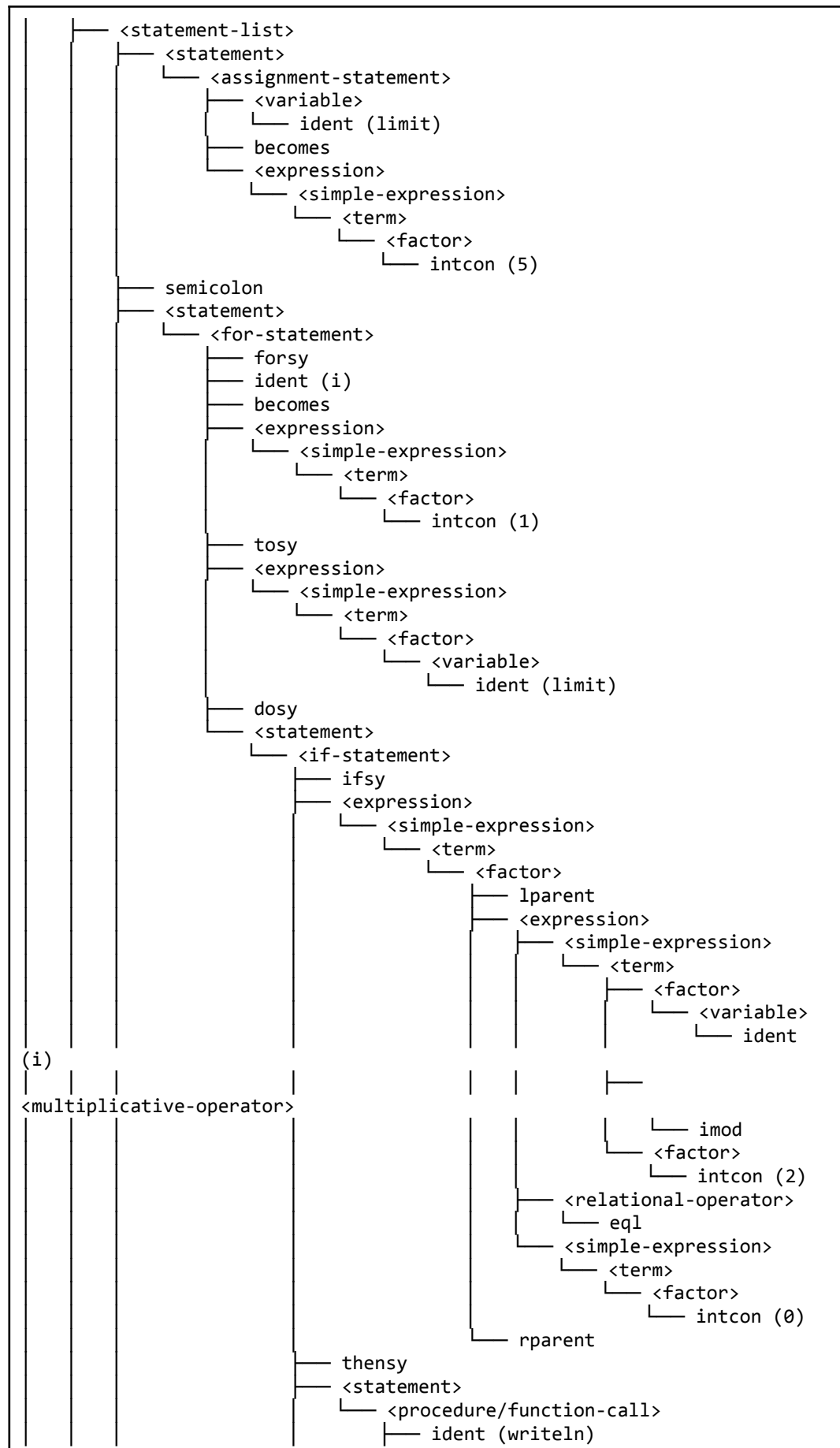
ident (limit)
minus
intcon (1)
semicolon
untilsy
ident (limit)
eq1
intcon (0)
semicolon
casesy
ident (choices)
ofsy
charcon ('A')
colon
ident (level)
becomes
intcon (100)
semicolon
charcon ('B')
colon
ident (level)
becomes
intcon (50)
semicolon
charcon ('C')
colon
ident (level)
becomes
intcon (10)
endsy
semicolon
semicolon
endsy
period

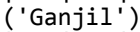
```

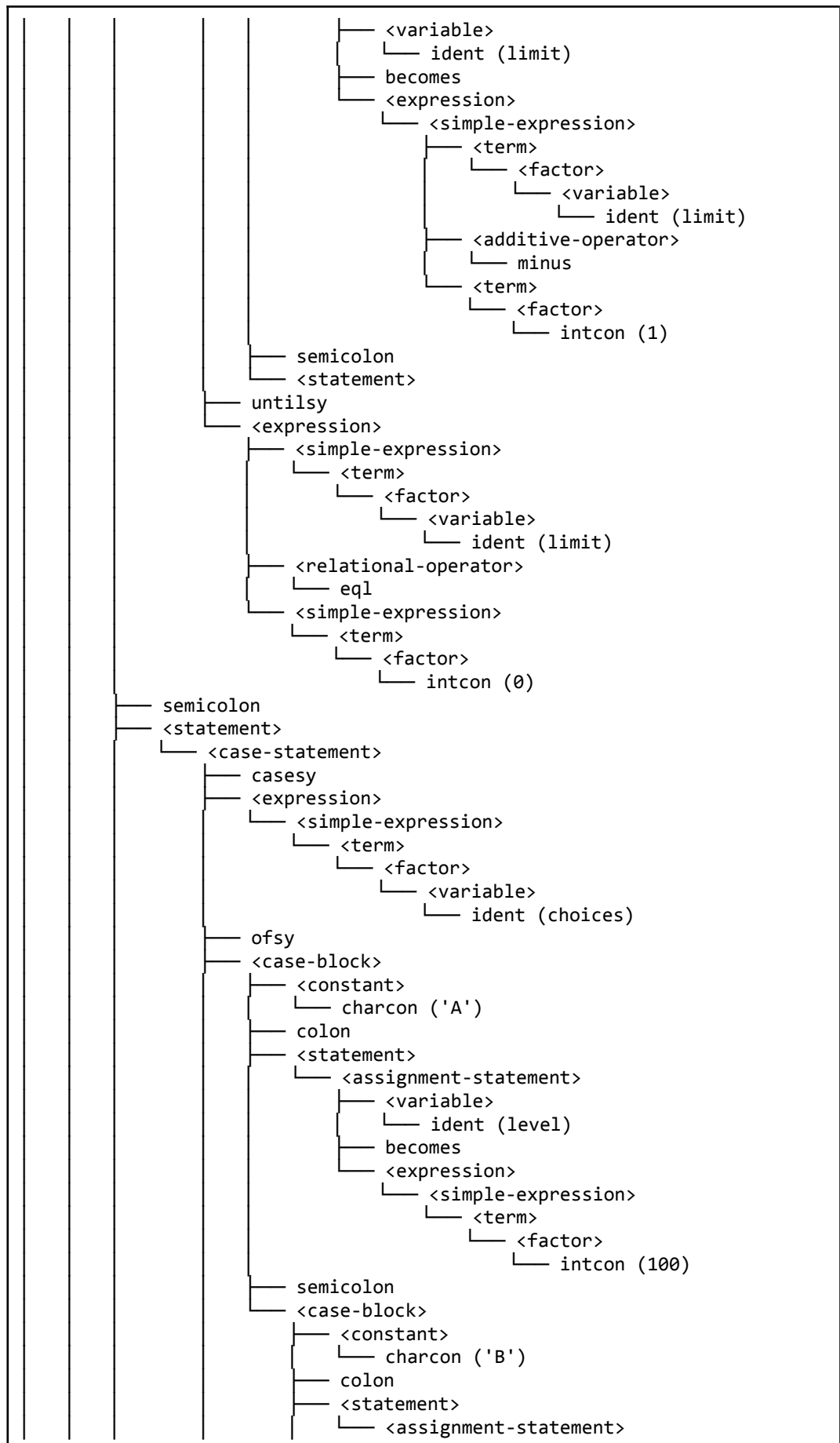
```

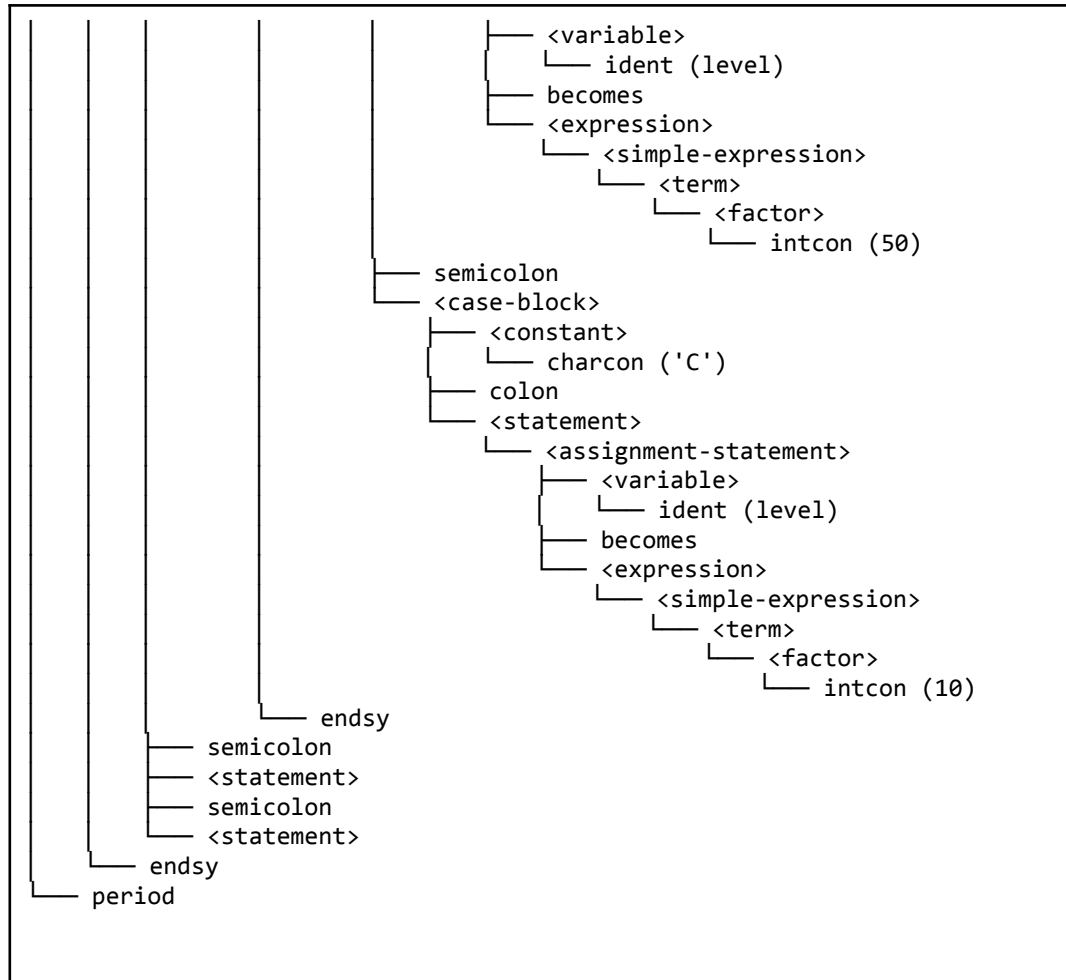
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (OddvsEven)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (i)
│       │   ├── comma
│       │   ├── ident (limit)
│       │   ├── comma
│       │   └── ident (level)
│       ├── colon
│       ├── <type>
│       │   └── ident (Integer)
│       ├── semicolon
│       ├── <identifier-list>
│       │   └── ident (choices)
│       ├── colon
│       ├── <type>
│       │   └── ident (Char)
│       └── semicolon
└── <compound-statement>
    └── beginsy

```







Bukti screenshot:

<pre> program OddvsEven; var i, limit, level: Integer; choices: Char; begin limit := 5; for i := 1 to limit do if (i mod 2 = 0) then writeln('Genap') else writeln('Ganjil'); while i < 10 do i := i + 1; repeat limit := limit - 1; until limit = 0; case choices of 'A': level := 100; 'B': level := 50; 'C': level := 10 end; ; end. </pre>	<pre> 1 <program> 2 <program-header> 3 programs 4 ident (OddvsEven) 5 semicolon 6 <declaration-part> 7 <var-declaration> 8 varsy 9 <identifier-list> 10 ident (i) 11 comma 12 ident (limit) 13 comma 14 ident (level) 15 colon 16 <type> 17 ident (Integer) 18 semicolon 19 <identifier-list> 20 ident (choices) 21 colon 22 <type> 23 ident (Char) 24 semicolon 25 <compound-statement> 26 beginsy 27 <statement-list> 28 <statement> 29 <assignment-statement> 30 <variable> 31 ident (limit) 32 becomes 33 <expression> 34 <simple-expression> 35 <term> </pre>
---	--

Gambar 4. Input dan Output Test Case 4

3.5. Test Case 5

program Testing;

```

var
    hasil, i, temp, trial, a, b, c: Integer;
    valid: Boolean;

procedure calculate(a, b: integer);
var
    temp, i, batas: Integer;
begin
    temp := a mod b;
    batas := 20;
    for i := temp to batas do
        if(i mod a+b == 0) then
            a := a + B;
    writeln(a)
end;

begin
    for i := 0 to trial do
        calculate(hasil, temp);
    a := 10;
    b := 3;

    c := a div b + a mod b * 5 - (a + 2) * (b - 1);
    valid := (a > b) AND (c <> 0) OR NOT (a <= 5);

    if valid then
        c := c * 1;
end.

```

```

programsy
ident (Testing)
semicolon
varsy
ident (hasil)
comma
ident (i)
comma
ident (temp)
comma
ident (trial)
comma
ident (a)
comma
ident (b)
comma
ident (c)
colon
ident (Integer)
semicolon
ident (valid)
colon
ident (Boolean)
semicolon
proceduresy
ident (calculate)
lparent
ident (a)
comma
ident (b)

```

```
colon
ident (integer)
rparent
semicolon
varsy
ident (temp)
comma
ident (i)
comma
ident (batas)
colon
ident (Integer)
semicolon
beginsy
ident (temp)
becomes
ident (a)
imod
ident (b)
semicolon
ident (batas)
becomes
intcon (20)
semicolon
forsy
ident (i)
becomes
ident (temp)
tosy
ident (batas)
dosy
ifsy
lparent
ident (i)
imod
ident (a)
plus
ident (b)
eq1
intcon (0)
rparent
thensy
ident (a)
becomes
ident (a)
plus
ident (B)
semicolon
ident (writeln)
lparent
ident (a)
rparent
endsy
semicolon
beginsy
forsy
ident (i)
becomes
intcon (0)
tosy
ident (trial)
```

```
dosy
ident (calculate)
lparent
ident (hasil)
comma
ident (temp)
rparent
semicolon
ident (a)
becomes
intcon (10)
semicolon
ident (b)
becomes
intcon (3)
semicolon
ident (c)
becomes
ident (a)
idiv
ident (b)
plus
ident (a)
imod
ident (b)
times
intcon (5)
minus
lparent
ident (a)
plus
intcon (2)
rparent
times
lparent
ident (b)
minus
intcon (1)
rparent
semicolon
ident (valid)
becomes
lparent
ident (a)
gtr
ident (b)
rparent
andsy
lparent
ident (c)
neq
intcon (0)
rparent
orsy
notsy
lparent
ident (a)
leq
intcon (5)
rparent
semicolon
```

```

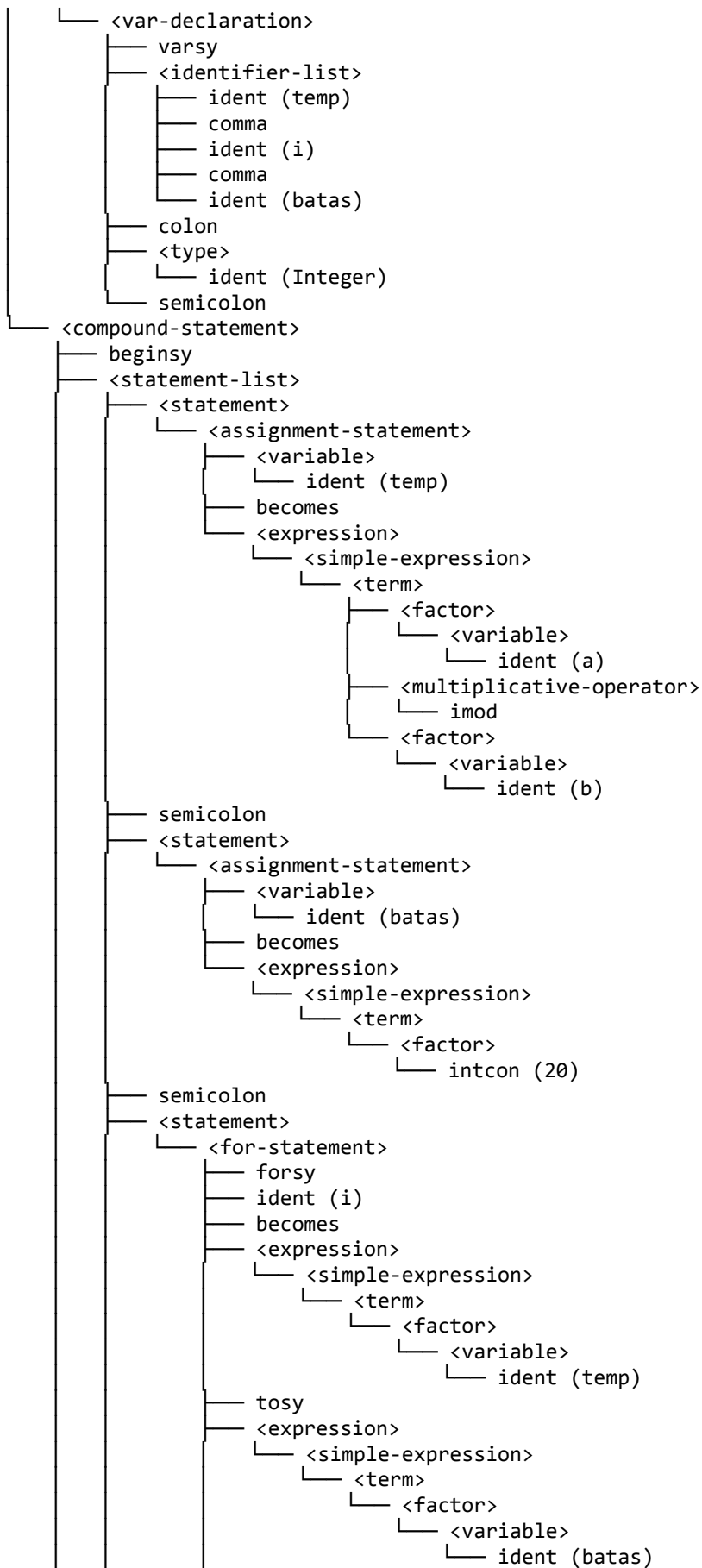
ifsy
ident (valid)
thensy
ident (c)
becomes
ident (c)
times
intcon (1)
semicolon
endsy
period

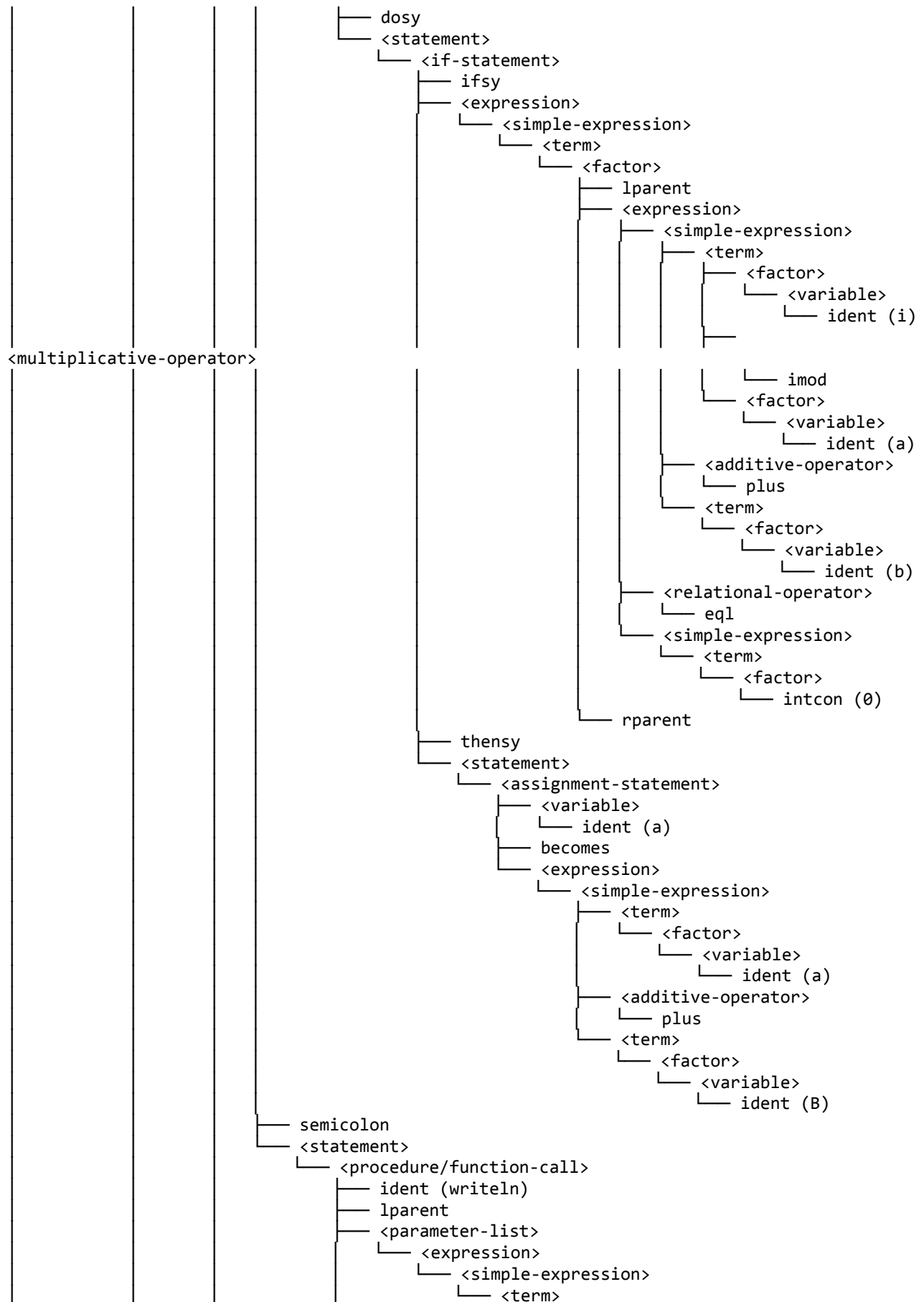
```

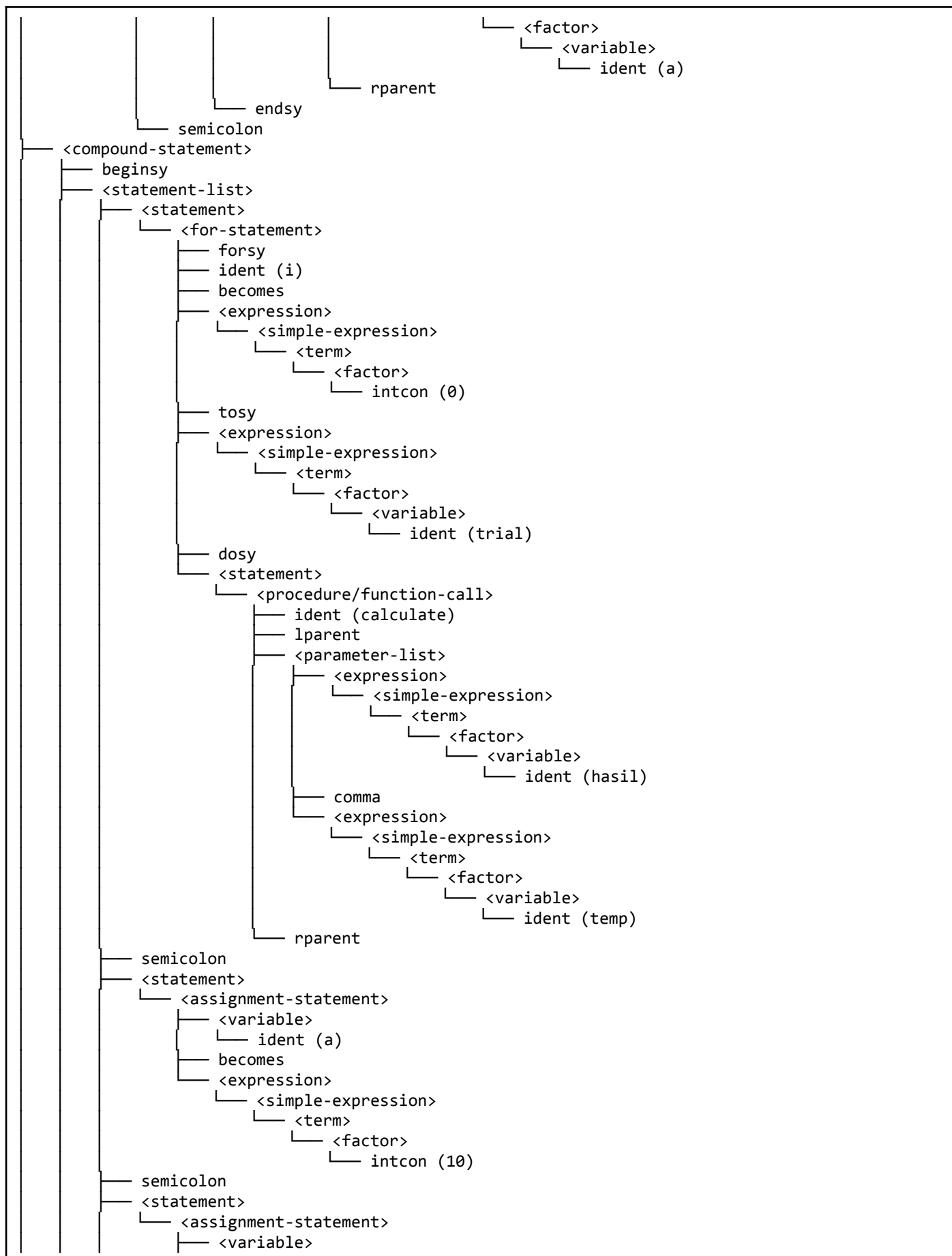
```

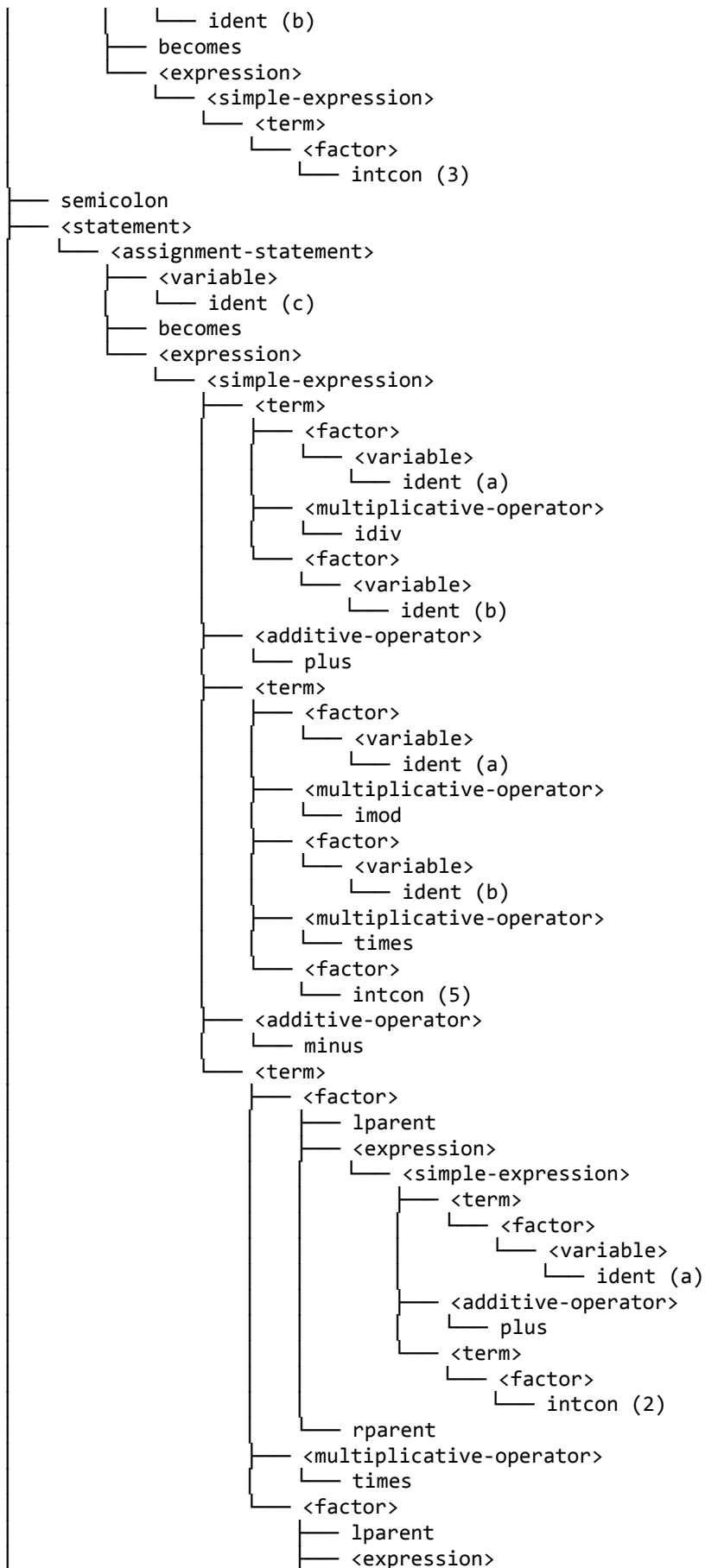
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Testing)
│   └── semicolon
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   ├── ident (hasil)
│   │   │   ├── comma
│   │   │   ├── ident (i)
│   │   │   ├── comma
│   │   │   ├── ident (temp)
│   │   │   ├── comma
│   │   │   ├── ident (trial)
│   │   │   ├── comma
│   │   │   ├── ident (a)
│   │   │   ├── comma
│   │   │   ├── ident (b)
│   │   │   ├── comma
│   │   │   └── ident (c)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (Integer)
│   │   ├── semicolon
│   │   ├── <identifier-list>
│   │   │   └── ident (valid)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (Boolean)
│   │   └── semicolon
│   └── <array-type>
│       ├── <array-type>
│       │   ├── proceduresy
│       │   ├── ident (calculate)
│       │   ├── <array-type>
│       │   │   ├── lparent
│       │   │   ├── <parameter-group>
│       │   │   │   ├── <identifier-list>
│       │   │   │   │   ├── ident (a)
│       │   │   │   │   ├── comma
│       │   │   │   │   └── ident (b)
│       │   │   │   ├── colon
│       │   │   │   └── ident (integer)
│       │   │   └── rparent
│       │   ├── semicolon
│       │   ├── <array-type>
│       │   └── <declaration-part>

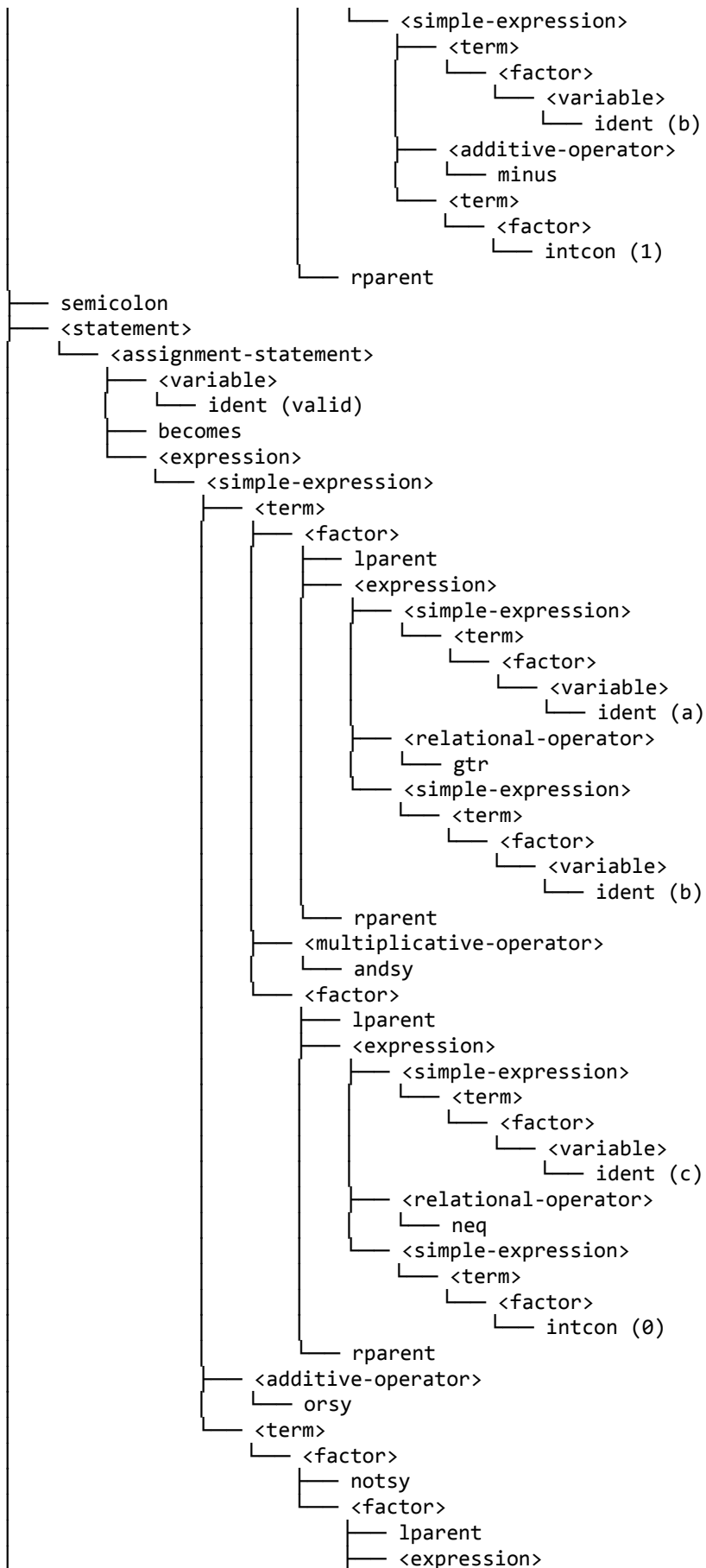
```

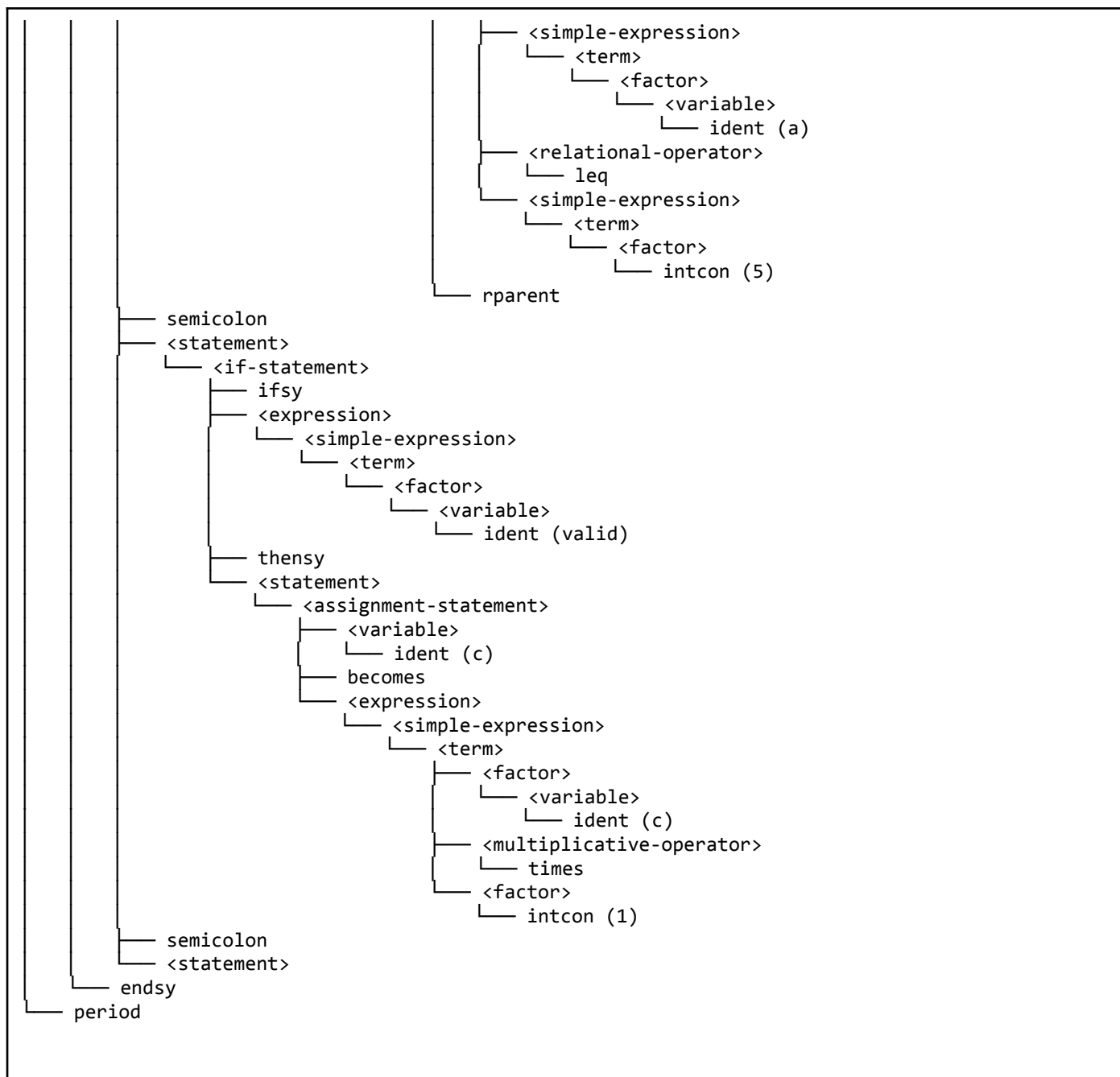













Bukti screenshot:


```

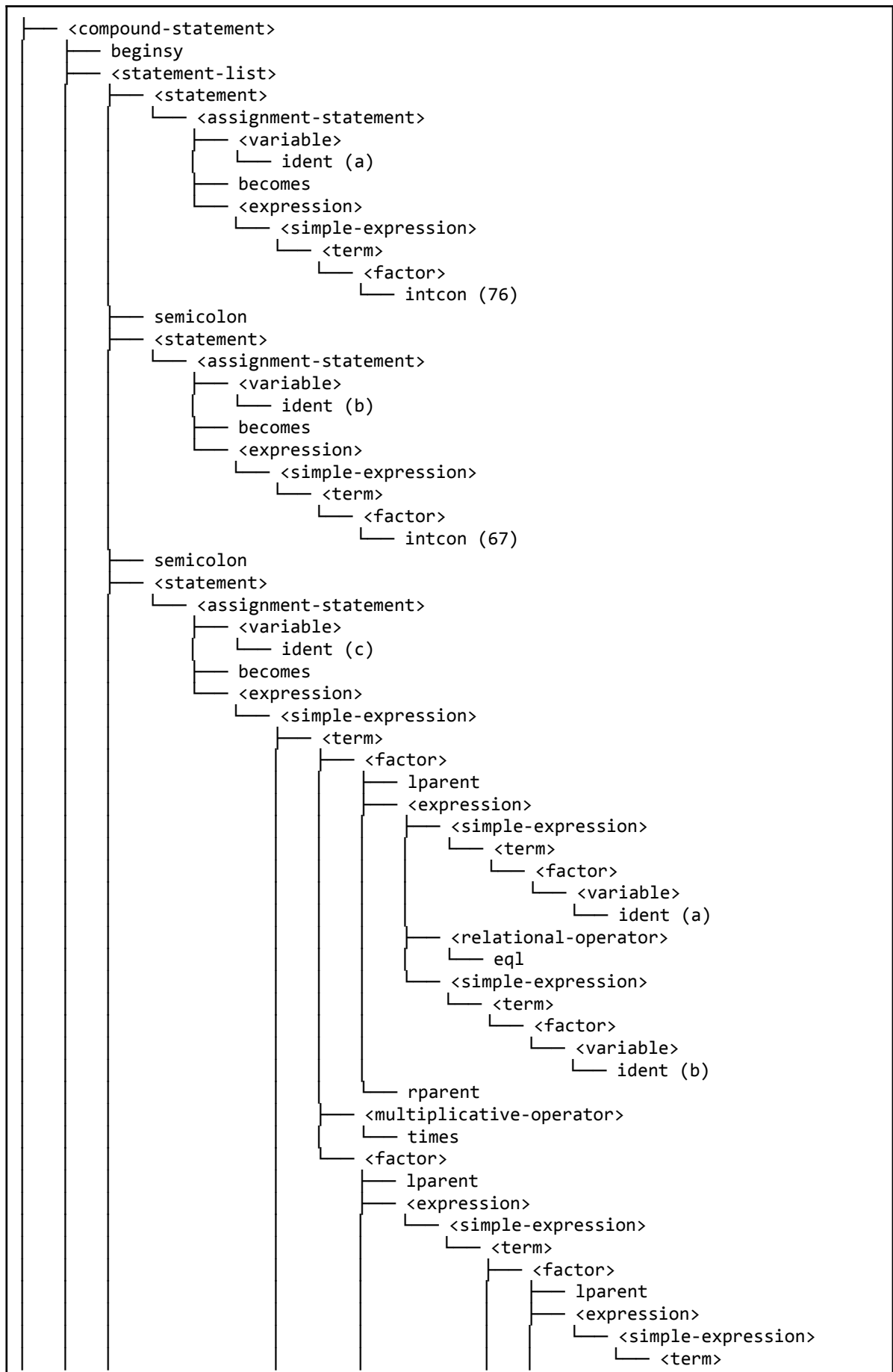
times
lparent
lparent
ident (a)
rdiv
ident (b)
rparent
imod
lparent
ident (a)
idiv
ident (b)
rparent
rparent
plus
lparent
ident (a)
neq
ident (b)
rparent
times
lparent
lparent
ident (a)
plus
ident (b)
rparent
idiv
lparent
ident (a)
minus
ident (b)
rparent
rparent
semicolon
ident (writeln)
lparent
ident (c)
rparent
semicolon
endsy
period

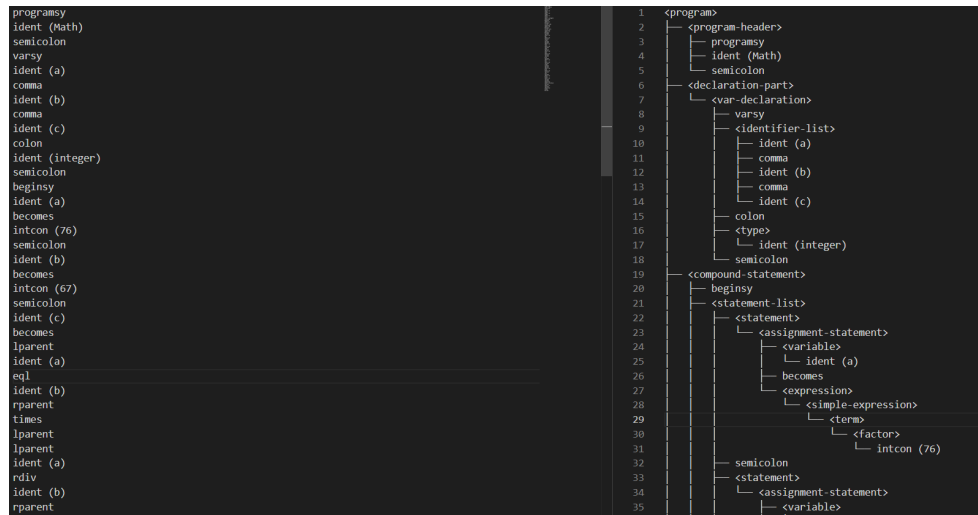
```

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Math)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   ├── ident (b)
│       │   ├── comma
│       │   └── ident (c)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon

```



Gambar 6. Input dan Output Test Case 6

3.7. Test Case 7

```

program Circle;

var
  Pi: real;
  r, c, s: real;
  identIniSangatPanjang24tetapiTetapValid: integer;

begin
  Pi := 3.14;
  r := 5;

  c := 2 * Pi * r;
  s := Pi * r * r;
  println('Keliling: ', c);
  println('Luas: ', s);
  println(identIniSangatPanjang24tetapiTetapValid);
end.

```

```

programsy
ident (Circle)
semicolon
varsy
ident (Pi)
colon
ident (real)
semicolon
ident (r)
comma
ident (c)
comma
ident (s)
colon
ident (real)
semicolon
ident (identIniSangatPanjang24tetapiTetapValid)
colon
ident (integer)

```

```

semicolon
beginsy
ident (Pi)
becomes
realcon (3.14)
semicolon
ident (r)
becomes
intcon (5)
semicolon
ident (c)
becomes
intcon (2)
times
ident (Pi)
times
ident (r)
semicolon
ident (s)
becomes
ident (Pi)
times
ident (r)
times
ident (r)
semicolon
ident (println)
lparent
string ('Keliling: ')
comma
ident (c)
rparent
semicolon
ident (println)
lparent
string ('Luas: ')
comma
ident (s)
rparent
semicolon
ident (println)
lparent
ident (identIniSangatPanjang24tetapiTetapValid)
rparent
semicolon
endsy
period

```

```

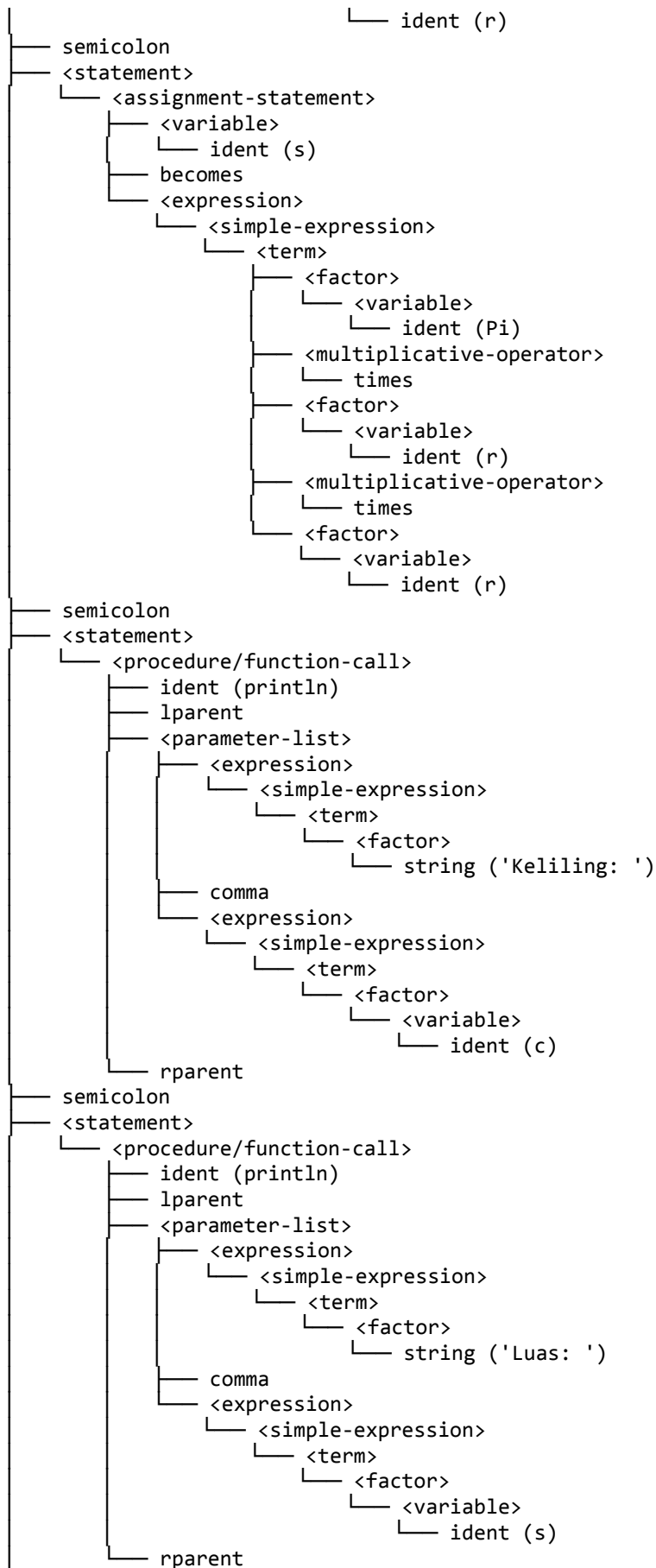
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Circle)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (Pi)
│       │   └── colon
│       ├── <type>
│       └── ident (real)

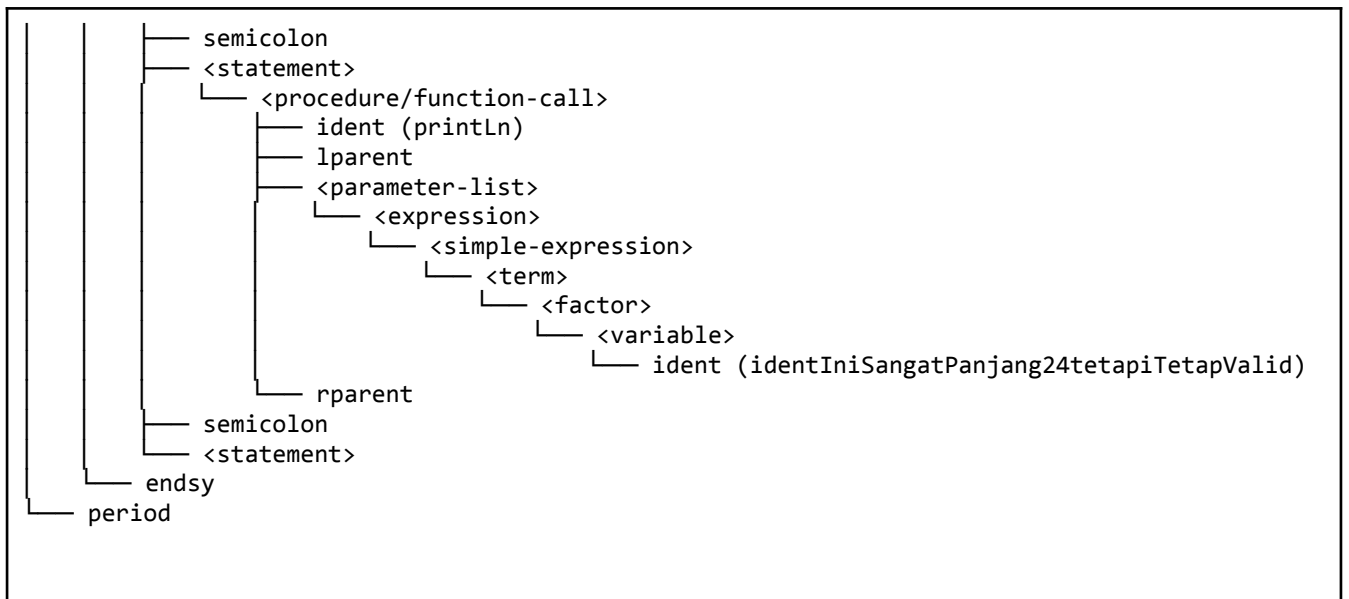
```

```

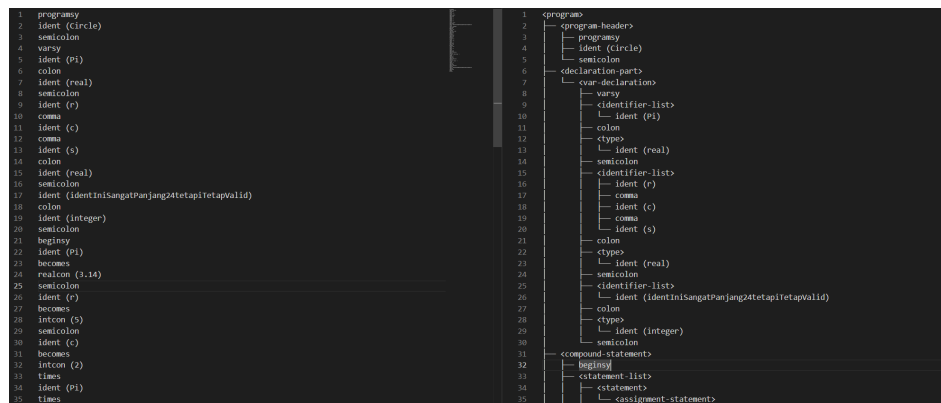
— semicolon
— <identifier-list>
  — ident (r)
  — comma
  — ident (c)
  — comma
  — ident (s)
— colon
— <type>
  — ident (real)
— semicolon
— <identifier-list>
  — ident (identIniSangatPanjang24tetapiTetapValid)
— colon
— <type>
  — ident (integer)
— semicolon
— <compound-statement>
  — beginsy
  — <statement-list>
    — <statement>
      — <assignment-statement>
        — <variable>
          — ident (Pi)
        — becomes
        — <expression>
          — <simple-expression>
            — <term>
              — <factor>
                — realcon (3.14)
      — semicolon
    — <statement>
      — <assignment-statement>
        — <variable>
          — ident (r)
        — becomes
        — <expression>
          — <simple-expression>
            — <term>
              — <factor>
                — intcon (5)
      — semicolon
    — <statement>
      — <assignment-statement>
        — <variable>
          — ident (c)
        — becomes
        — <expression>
          — <simple-expression>
            — <term>
              — <factor>
                — intcon (2)
              — <multiplicative-operator>
                — times
              — <factor>
                — <variable>
                  — ident (Pi)
              — <multiplicative-operator>
                — times
              — <factor>
                — <variable>

```





Bukti screenshot



Gambar 7. Input dan Output Test Case 7

3.8. Test Case 8

```

program Sky;

var
  s, c: string;

begin
  s := 'green';
  c := 'blue';
  if s == c then {print when sky is blue}
  begin
    writeln('sky is blue');
  end.
end.

programsy
ident (Sky)
semicolon
  
```



```

varsy
ident (s)
comma
ident (c)
colon
ident (string)
semicolon
beginsy
ident (s)
becomes
string ('green')
semicolon
ident (c)
becomes
string ('blue')
semicolon
ifsy
ident (s)
eq1
ident (c)
thensy
comment (print when sky is blue)
beginsy
ident (writeln)
lparent
string ('sky is blue')
rparent
semicolon
endsy
period
endsy
period

```

Syntax error:10:5: unexpected token beginsy ("begin"), expected casesy|elsesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy

Bukti *screenshot*:

```

test > Lexer > tc8.txt_tokenize
1  programsy
2  ident (Sky)
3  semicolon
4  varsy
5  ident (s)
6  comma
7  ident (c)
8  colon
9  ident (string)
10 semicolon
11 beginsy
12 ident (s)
13 becomes
14 string ('green')
15 semicolon
16 ident (c)
17 becomes
18 string ('blue')
19 semicolon
20 ifsy
21 ident (s)
22 eql
23 ident (c)
24 thensy
25 comment (print when sky is blue)
26 beginsy
27 ident (writeln)
28 lparent
29 string ('sky is blue')
30 rparent
31 semicolon
32 endsy
33 period
34 endsy
35 period

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\kdste\Documents\Files\FileDaro\sem4\TBFO\tubes\TBF-Tubes-IF2224-2026> wsl
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc2.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc3.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc4.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc5.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc6.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc7.txt
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc8.txt
Syntax error:10:5: unexpected token beginsy ("begin"), expected casesy|elsesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy
kdste@KD24: /mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc9.txt
^[[Syntax error:10:5: unexpected token beginsy ("begin"), expected casesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy

```

Gambar 8. Input dan Output Test case 8

3.9. Test Case 9

```

program Star;

var
    i, n: integer;

begin
    i := 1;
    n := 5;
    while i <= n do
        begin
            writeln('*');
            i := i + 1;
        end.
    end.
end.

```

```

programsy
ident (Star)
semicolon
varsy
ident (i)
comma
ident (n)

```

```
colon
ident (integer)
semicolon
beginsy
ident (i)
becomes
intcon (1)
semicolon
ident (n)
becomes
intcon (5)
semicolon
whilesy
ident (i)
leq
ident (n)
dosy
beginsy
ident (writeln)
lparent
charcon ('*')
rparent
semicolon
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon
endsy
period
endsy
period
```

```
^[[ASyntax error:10:5: unexpected token beginsy ("begin"), expected casesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy
```

Bukti *screenshot*

```
tc1.txt x tc9.txt_tokenize U x
test > Lexer > tc9.txt_tokenize
1  programsy
2  ident (Star)
3  semicolon
4  varsy
5  ident (i)
6  comma
7  ident (n)
8  colon
9  ident (integer)
10 semicolon
11 beginsy
12 ident (i)
13 becomes
14 intcon (1)
15 semicolon
16 ident (n)
17 becomes
18 intcon (5)
19 semicolon
20 whilesy
21 ident (i)
22 leq
23 ident (n)
24 dosy
25 beginsy
26 ident (writeln)
27 lparent
28 charcon ('*')
29 rparent
30 semicolon
31 ident (i)
32 becomes
33 ident (i)
34 plus
35 intcon (1)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\kdste\Documents\Files\FileDaro\sem4\TBFO\tubes\TBF-Tubes-IF2224-2026> wsl
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc2.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc3.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc4.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc5.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc6.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc7.txt
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc8.txt
Syntax error:10:5: unexpected token beginsy ("begin"), expected casesy|elsesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy
kdste@KD24:/mnt/c/Users/kdste/Documents/Files/FileDaro/sem4/TBFO/tubes/TBF-Tubes-IF2224-2026$ ./bin/program.exe tc9.txt
^[[Syntax error:10:5: unexpected token beginsy ("begin"), expected casesy|endsy|forsy|ident|ifsy|repeatsy|semicolon|whilesy
```

Gambar 9. Input dan Output Test Case 9

4. Kesimpulan dan Saran

4.1. Kesimpulan

Dalam Milestone 2 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata ini, kami berhasil mengimplementasikan *syntax analyzer* untuk bahasa pemrograman Arion menggunakan algoritma *recursive descent parsing* sebagai lanjutan dari implementasi *lexical analyzer* pada *milestone* sebelumnya. Proses algoritma yang diimplementasikan melibatkan pembangunan *parse tree* terhadap token yang sudah dihasilkan oleh *lexer* berdasarkan aturan produksi yang ada sehingga diketahui apakah susunan token yang dihasilkan *lexer* sudah sesuai *syntax* yang diharapkan atau belum dan dapat menentukan letak kesalahan *syntax* pada kode program awal. Pengimplementasian tersebut merupakan salah satu contoh penerapan salah satu algoritma *parsing* yang dipelajari di mata kuliah Teori Bahasa Formal dan Otomata.

4.2. Saran

Kami sangat berterima kasih atas kerja luar biasa dari asisten dalam perancangan spesifikasi milestone dua tugas besar ini, keseragaman input/output dalam tiap tahap membuat kerja kami menjadi sangat modular dan tidak terlalu saling bergantung terhadap milestone sebelumnya. Kami menyarankan perubahan atau revisi pada spesifikasi tidak dituliskan terlalu lama sehingga mencegah kemungkinan perubahan kode atau kelalaian kelompok dalam mengubah kode yang sudah tertulis detail rincinya, guna mempermudah pengerjaan tugas ini dan membantu kami fokus mengerjakan aspek tugas yang lebih penting.

5. Lampiran

5.1. Github

<https://github.com/Naufal-Pinasthika/TBF-Tubes-IF2224-2026>

5.2. Tabel Pembagian Tugas

Nama	NIM	Workload	Persentase
Faiq Azzam Nafidz	13524003	Membuat struktur kelas terkait parser, membuat desain awal parser, membuat fungsi produksi 1–10 + 25–27, membuat fungsi pencetak <i>parse tree</i> , membuat mekanisme <i>error handling</i> , membuat laporan (perancangan & implementasi).	35%
Anindya Naufal Pinasthika	13524013	Membuat fungsi produksi 34–43, membuat fitur pemilihan proses program (antara <i>lexer</i> atau <i>parser</i>), memodifikasi README.md, membuat laporan (teori singkat).	25%
Mahmudia Kimdaro Amin	13524083	Membuat fungsi produksi 21–24 + 28–33, melakukan pengujian program, membuat laporan (pengujian).	20%
Jingglang Galih Rinenggan	13524095	Membuat fungsi produksi 11–20, membuat laporan (kesimpulan dan saran).	20%

6. Referensi

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson. Tersedia di: [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf) (Diakses pada 2 April 2026)
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson/Addison-Wesley.